

Competitive Programming Handbook

Ing. Felipe Rafael Focil Mendoza
Co-editores: Fernanda Avedaño y Emiliano Castaños

Versión 2.0
April 27, 2026

Contents

0.1	Referencias	7
1	Edición y Colaboradores	7
1.0.1	Autor y Editor Principal	7
1.0.2	Co-editores y colaboradores	7
1.1	Agradecimientos	7
2	Prefacio	8
3	Introducción a la Programación Competitiva	9
3.1	Veredictos	9
3.2	Compilación	9
3.3	Lectura	9
3.3.1	Strings con espacios	9
3.3.2	End of File	9
3.3.3	Archivos	10
3.4	Atajos	10
3.4.1	Macros	10
3.4.2	Using	10
3.4.3	Fast Input Normalizer	10
3.4.4	Underscore main	10
3.4.5	Libreria	10
4	Plantilla	11
4.1	Precision	11
4.1.1	Enteros	11
4.1.2	Decimales	11
4.2	Tiempo de Complejidad	12
4.3	Manipulación de Bits	12
4.3.1	Operadores AND, OR y XOR	12
4.4	Bitset	14
4.4.1	Entero a binario	14
4.4.2	Binario a entero	14
4.4.3	Mascara de bits	14
4.4.4	Propiedades	15
5	Ordenamiento	15
5.1	Operadores de Comparación	15
6	Busqueda Binaria	17

6.1	Busqueda Binaria con STL	18
6.1.1	Verificar si existe	18
6.1.2	Encontrar indice	18
6.1.3	Primer valor que sea menor	18
6.1.4	Primer valor que sea mayor	19
7	Busqueda Ternaria	19
8	Combinatoria	20
8.1	Definición general	20
8.2	Fórmula Recursiva	20
8.3	Obtener todas combianciones	20
9	Permutaciones	22
9.1	Definición general	22
9.2	Forma recursiva	22
9.3	Obtener todas combianciones	22
10	Propiedades de Combinatoria y Permutaicones	23
10.0.1	Teorema Binomial	23
10.1	Palo de Hockey	23
10.2	Números de Catalán	23
11	Principio de Inclusión - Exclusión	24
11.1	Dos conjuntos	24
11.2	Tres conjuntos	24
11.3	Caso general	24
11.4	Derangements (Desarreglos)	24
11.5	Burnside's Lemma	24
12	Exponenciación Binaria	25
13	Aritmética Modular	25
13.0.1	Operaciones mediante aritmética modular	25
14	Teoría de Números	26
14.1	Números Primos y Factores	26
14.1.1	Cantidad de factores	27
14.2	Suma de factores	28
14.2.1	Lista de factores	29
14.3	Pruebas de primalidad	29
14.3.1	Algoritmos probabilisticos	29

14.3.2	Primos como diferencia de cuadrados	31
14.4	Computar números primos	31
14.4.1	Criba de Erastotenes	31
14.4.2	Criba Lineal	32
14.4.3	Contar número de primos en un rango	32
14.4.4	Encontrar primos en un rango	33
14.5	Algoritmo de Euclides	33
14.5.1	Máximo Común Divisor	33
14.5.2	Máximo Común Multiplo	33
14.5.3	Numeros Coprimos	34
14.5.4	Funcion Totiente de Euler	34
14.5.5	Teorema de Euler	35
14.5.6	Pequeño Teorema de Fermat	35
14.6	Inversos Multiplicativos Modulares	35
14.7	Ecuaciones Diofánticas	36
14.7.1	Teorema Chino del Resto	37
14.8	MEX	38
15	Juegos NIM	38
15.1	Nim Sum	38
15.2	Movimiento óptimo	38
15.3	Teorema de Sprague–Grundy	38
15.4	Números de Grundy	39
16	Estructuras de Datos	39
16.1	Arreglos Dinámicos	39
16.1.1	Vectores	39
16.1.2	Iteradores y rangos	40
16.1.3	Deque	41
16.1.4	Stack	41
16.1.5	Queue	41
16.2	Estructuras Set	41
16.2.1	Sets y multisets	41
16.2.2	Maps	43
16.2.3	Priority Queues (Colas de Prioridad)	44
16.2.4	Policy-Based Sets (Conjuntos Basados en Políticas)	45
17	Monotonic Stack	45
18	Programación Dinamica	46

18.0.1	Patrones Tipicos	47
18.0.2	Lineal (1D DP)	47
18.1	Conjuntos (Bitmask DP)	47
18.2	Intervalos	48
18.3	Longest Increasing Subsequence (LIS)	48
18.4	Paths in a Grid	49
18.5	Knapsack	49
18.5.1	Reconstruir Knapsack	50
18.6	Counting Tilings (Grid DP)	52
19	Grafos	52
19.1	Terminologia	52
19.2	Representaciones de Grafos	53
19.2.1	Lista de Adyacencia	53
19.2.2	Matiz de Adyacencia	54
19.2.3	Lista de Aristas	54
19.3	Recorridos de Grafos	54
19.3.1	Depth First Search (DFS)	54
19.4	Breadth First Search (BFS)	55
19.5	Comprobaciones	56
19.5.1	Conectividad	56
19.5.2	Ciclos	57
19.5.3	Bipartito	58
19.6	Camino más corto	59
19.6.1	Dijkstra	59
19.6.2	Bellman-Ford	60
19.6.3	Floyd-Warshall	60
19.6.4	Topological Sort	60
19.7	Dynamic Programming for Graphs	62
19.8	Succesor Graphs	63
19.8.1	Deteccion de Ciclos	64
19.8.2	Puentes	65
20	Minimum Spanning Tree	65
20.0.1	Union-Find / Disjoint Sets	66
20.0.2	Kruskal	66
20.0.3	Prim	68
20.0.4	Strong Connectivity	69
20.0.5	Kosaraju	69

20.0.6 2 SAT	70
20.0.7 2-SAT	70
20.0.8 2-SAT	70
20.0.9 Eulerian Paths and Circuits	71
20.0.10 Hamiltonian Path	73
20.0.11 De Bruijn Sequences	75
20.0.12 Knight's Tour	76
21 Flujos	77
21.1 Red de Flujos	77
21.2 Grafo Residual	77
21.3 Ford-Fulkerson	78
21.4 Edmonds-Karps	79
21.5 Teorema del Corte Minimo	80
21.5.1 Disjoint Paths	81
21.6 Bipartite Matching	81
21.7 Puentes	83
21.8 Puntos de articulacion	83
21.9 Patrones y Diseño de Algoritmos	84
21.9.1 Hamming Distance	84
21.9.2 Reachability in Graphs	85
21.9.3 Counting Subgrids	85
21.9.4 Two Pointers Method	86
21.9.5 Nearest Smaller Element (Amortized Stack)	86
21.9.6 Sliding Window Minimum	86
21.9.7 Minimizing Sums	87
22 Range Queries	88
22.1 Segment Tree	88
22.2 Sparse Table	89
22.3 Fenwick Tree	90
22.4 Index Compression	91
23 Arboles	91
23.1 Binary Tree Traversals	91
24 Calculating Tree Diameter	93
24.1 All longest paths	94
24.2 Lowest Common Ancestor (LCA)	95
24.3 Balanced Factor	95

25 Trie	96
26 Rootear un árbol desde el centro	98
27 Identificar dos árboles isomorficos	99
28 Algoritmo de Tarjan para Componentes Fuertemente Conexas	100
28.1 Sucesiones	101
28.2 Sumatorias	101
28.3 Combinatoria	101
28.4 Números Especiales	102
28.4.1 Divisibilidad	104
29 Meet in the Middle	105
30 Algoritmo de Mo	105
31 Algoritmo de Kadane	107
32 Binary Lifting	108
33 Knuth-Morris-Pratt KMP	109
34 Algoritmo de Manacher	110
35 Longest Common Subsequence (LCS):	110
36 Rootear un árbol	111
37 Identificar dos árboles isomorficos	113
38 Algoritmo de Tarjan para Componentes Fuertemente Conexas	114

Aviso de derechos de autor y uso

© 2025 Felipe Focil. Todos los derechos reservados.

Este libro y todo su contenido —incluyendo texto, diagramas, ejemplos de código, explicaciones teóricas y estructuras organizativas— son propiedad intelectual de su autor.

Se autoriza la lectura, copia, impresión y distribución gratuita únicamente con permiso expreso del autor. El autor puede otorgar copias o versiones digitales e impresas de este material de manera libre y sin costo, a personas o grupos que él elija.

Queda estrictamente prohibida la venta, redistribución masiva o publicación en medios públicos (impresos o digitales) sin consentimiento previo y por escrito del autor. Cualquier modificación, traducción o adaptación deberá conservar esta nota y reconocer la autoría original.

Este material puede compartirse o imprimirse de manera gratuita solo si el autor lo autoriza directamente.

0.1 Referencias

El contenido de este libro se inspira parcialmente en materiales ampliamente reconocidos en la comunidad de programación competitiva, entre ellos:

Antti Laaksonen (2020). Guide to Competitive Programming: Learning and Improving Algorithms Through Contests, Third Edition. Springer.

CP-Algorithms. <https://cp-algorithms.com/> — Una referencia abierta y colaborativa mantenida por la comunidad de programación competitiva.

1 Edición y Colaboradores

1.0.1 Autor y Editor Principal

- Ing. Felipe Rafael Focil Mendoza

1.0.2 Co-editores y colaboradores

- Fernanda Avedaño
- Emiliano Castaños

1.1 Agradecimientos

A los estudiantes que han utilizado versiones previas del Handbook y ofrecido retroalimentación valiosa, ayudando a refinar su contenido y estructura.

Al Mtro. Aldo Tobón, por sentar las bases de este proyecto con su Píldora de Programación, que evolucionaría en el Handbook actual, y por su labor como coach e inspiración constante.

Con especial gratitud al Dr. Horacio Jarquín, cuyo acompañamiento, guía y exigencia académica han sido fundamentales para la formación del autor, tanto en la programación competitiva como en el pensamiento analítico. Su compromiso, disciplina y pasión por enseñar han dejado una huella profunda en esta obra y en quienes han aprendido bajo su dirección.

A todos los participantes, entrenadores y colegas que continúan impulsando la programación competitiva como un espacio de aprendizaje, colaboración y crecimiento.

—Felipe Focil

“La suerte solo beneficia a la mente preparada.”

2 Prefacio

El objetivo del Competitive Programming Handbook es ofrecer una recopilación práctica de código y tópicos comunes en concursos de programación, especialmente en el contexto de la ICPC (International Collegiate Programming Contest).

A menudo, los estudiantes poseen el conocimiento teórico necesario, pero bajo las condiciones de alta presión típicas de la competencia, la implementación puede tornarse imprecisa o lenta.

Este handbook busca servir como una guía de referencia rápida y confiable, reuniendo plantillas, técnicas y algoritmos frecuentemente utilizados por equipos universitarios durante el entrenamiento y las competencias.

3 Introducción a la Programación Competitiva

3.1 Veredictos

Código	Sinónimo	Posible explicación
CE	Compilation Error	Error de sintaxis, faltan librerías o tipos mal definidos.
RTE	Run Time Error	Acceso a memoria inválida, división entre cero, etc.
TLE	Time Limit Exceeded	Algoritmo ineficiente o bucle infinito.
MLE	Memory Limit Exceeded	Uso excesivo de arrays o estructuras grandes.
WA	Wrong Answer	Lógica incorrecta o caso no cubierto.
OLE	Output Limit Exceeded	Imprime más líneas de lo esperado.
PE	Presentation Error	Salto de línea o espacios extra.
AC	Accepted	El código fue aceptado.

3.2 Compilación

Sistema Operativo	Acción	Comando
Linux/Mac OS	Compilar	g++ -o problem> problem>.cpp
Windows	Compilar	g++ -o problem> problem>.cpp
Linux/Mac OS	Ejecutar	problem>
Windows	Ejecutar	problem> .exe

Para activar todas las advertencias o establecer una versión de C++ agrega los flags:

-Wall -Wextra -std=c++<version>:

```
1 g++ -Wall -Wextra -std=c++17 -o a problemaA.cpp
2 ./a
```

3.3 Lectura

3.3.1 Strings con espacios

```
1 string s;
2 getline(cin, s);
```

Limpiar buffer Si se alterna con lectura de enteros, en ocasiones se guardara el último salto de línea, por lo que será necesario limpiar el buffer.

```
1 cin >> edad;
2 cin.ignore(100, '\n');
3 getline(cin, nombre);
```

3.3.2 End of File

Si la cantidad de entrada es desconocida, leer hasta el final del archivo

```
1 while (cin>>x)
2 {
3     // code
4 }
```

3.3.3 Archivos

Redirigir la entrada y salida a archivos

```
1 freopen("input.in", "r", stdin);
2 freopen("output.out", "w", stdout);
```

3.4 Atajos

3.4.1 Macros

Remplaza el texto antes de compilar

```
1 #define PI 3.1416
2 #define SQUARE(x) ((x)*(x))
```

3.4.2 Using

Crea alias de tipos o espacios de nombres para escribir más corto y legible.

```
1 using ll = long long;
2 using vi = vector<int>;
3 using vvi = vector<vi>;
```

3.4.3 Fast Input Normalizer

Optimiza la velocidad de cin y cout. Evita la sincronización con stdio y acelera la entrada/salida.

```
1 ios::ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0)
```

3.4.4 Underscore main

Función donde va la lógica del problema, permite manejar varios casos sin ensuciar main().

```
1 int _main()
2 {
3     int a,b;
4     cin>>a>>b;
5     return a+b;
6 }
7
8 void main()
9 {
10    int t;
11    cin>>t;
12    while (t--)
13        cout<<_main*<<endl;
14    return 0;
15 }
```

3.4.5 Libreria

Incluye todas las librerías del Standard Library Template

```
1 #include <bits/stdc++.h>
```

4 Plantilla

```
1 #include <bits/stdc++.h>
2 #define FIN ios::ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0)
3 #define endl '\n'
4 #define forn(i,a,b) for(int i = int(a); i < int(b);i++)
5 #define pb push_back
6 #define all(v) v.begin(),v.end()
7 #define sz(a) int(a.size())
8 #define coutBool(a) cout << ((a) ? "YES" : "NO")<<endl
9 #define mp make_pair
10 #define fi first
11 #define se second
12 #define debug(x) cout<<#x<<" : "<<x<<endl
13 using namespace std;
14 using ll = long long;
15 using ull = unsigned long long;
16 using vi = vector<int>;
17 using vvi = vector<vi>;
18 using vl = vector<ll>;
19 using vvl = vector<vl>;
20 using vb = vector<bool>;
21 using vs = vector<string>;
22 using pi = pair<int, int>;
23 using u64 = uint64_t;
24
25 void _main() {
26
27 }
28 int main() {
29     FIN;
30     int t = 1;
31     cin >> t; // Elimina si solo tiene un caso
32     while (t--) {
33         _main();
34     }
35     return 0;
36 }
```

4.1 Precision

4.1.1 Enteros

Tipo	Base 2	Base 10	Base 10
bool	0 a 1	0 a 1	$1 * 10^0$
char	-128 a 127	-128 a 127	$2 * 10^2$
unsigned char	0 a 255	0 a 255	$2.55 * 10^2$
short	-32,768 a 32,767	-32,768 a 32,767	$3.28 * 10^4$
int	$-2^{31}a(2^{31} - 1)$	-2,147,483,648 a 2,147,483,647	$2.15 * 10^9$
long long	$-2^{63}a(2^{63} - 1)$	-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807	$9.22 * 10^{18}$
unsigned long long	0 a $(2^{64} - 1)$	0 a 18,446,744,073,709,551,615	$1.84 * 10^{19}$

4.1.2 Decimales

Tipo	Dígitos de precisión
float	7
double	15
long double	19

Establecer precision

```
1 double pi = 3.14159265358979323846;  
2 cout << fixed << setprecision(5) << pi << endl;
```

Compracion de decimales

```
1 if (abs(a-b) < 1e-9) // a == b
```

4.2 Tiempo de Complejidad

n	Posibles complejidades
$n \leq 10$	$O(n!)$, $O(n^7)$, $O(n^6)$
$n \leq 20$	$O(2^n * n)$, $O(n^5)$
$n \leq 80$	$O(n^4)$
$n \leq 400$	$O(n^3)$
$n \leq 7500$	$O(n^2)$
$n \leq 7 * 10^4$	$O(n\sqrt{n})$
$n \leq 5 * 10^5$	$O(n \log n)$
$n \leq 5 * 10^6$	$O(n)$
$n \leq 10^{18}$	$O(\log^2 n)$, $O(\log n)$, $O(1)$

4.3 Manipulación de Bits

4.3.1 Operadores AND, OR y XOR

Operación	Resultado
AND	Devuelve 1 solo si ambos bits son 1, 0 en otro caso
OR	Devuelve 1 si al menos un bit es 1, 0 si ambos son 0
XOR	Devuelve 1 si los bits son diferentes, 0 si son iguales
NOT	Invierte los bits: $1 \rightarrow 0$, $0 \rightarrow 1$
Desplazamiento izquierda	Mueve los bits a la izquierda, agregando ceros a la derecha
Desplazamiento derecha	Mueve los bits a la derecha, agregando ceros o unos según el tipo de dato

```
1 #include <iostream>  
2 using namespace std;  
3  
4 int main() {  
5     unsigned int a = 0b1100; // 12  
6     unsigned int b;  
7  
8     // Operaciones bit a bit  
9     b = a & 0b1010; // 0b1100 & 0b1010 = 0b1000 (8)  
10    cout << "AND: " << b << endl;  
11  
12    b = a | 0b1010; // 0b1100 | 0b1010 = 0b1110 (14)  
13    cout << "OR: " << b << endl;  
14  
15    b = a ^ 0b1010; // 0b1100 ^ 0b1010 = 0b0110 (6)  
16    cout << "XOR: " << b << endl;  
17  
18    b = ~a & 0b1111; // complemento limitado a 4 bits: ~0b1100 = 0b0011 (3)  
19    cout << "NOT: " << b << endl;  
20  
21    // Desplazamientos
```

```
22  b = a << 2; // 0b1100 << 2 = 0b110000 (48)
23  cout << "Shift left: " << b << endl;
24
25  b = a >> 2; // 0b1100 >> 2 = 0b0011 (3)
26  cout << "Shift right: " << b << endl;
27
28  return 0;
29 }
```

4.4 Bitset

Bitset es una estructura que permite una manipulación de bits más sencilla en cuestión de código y permite trabajar con cadenas de bits mayores a un long long int (64).

```
1 bitset<8> a;
2 bitset<5> b = 8;
3 // Establecer bits en posiciones específicas en 'a'
4 a[0] = 1;
5 a[2] = 1;
6 a[5] = 1;
7 // Contar los bits encendido en 'a'
8 int count = a.count();
9 // Comprobar si el bit en la posición '3' esta encendido
10 bool isSet = a.test(3);
11 // Establecer el bit en la posición '4' en '1'
12 a.set(4, true);
13 // Resetear el bit en la posición 2 a '0'
14 a.reset(2);
15 // Invertir el valor del bit en la posición '1'
16 a.flip(1);
17 string bitsetString = a.to_string();
18 bitset<8> anotherBitset("10101010");
19 bitset<8> c_and = a & b;
20 bitset<8> c_or = a | b;
21 bitset<8> c_xor = a ^ b;
22 bitset<8> c_not = ~a;
```

4.4.1 Entero a binario

```
1 string to_binary(int n) {
2     bitset<8> x(n);
3     return x.to_string();
4 }
```

4.4.2 Binario a entero

```
1 int binary_to_int(string s) {
2     return stoi(s, nullptr, 2);
3 }
```

4.4.3 Mascara de bits

Una máscara de bits de la forma $1 \ll k$ tiene un bit en 1 en la posición k y todos los demás bits en 0, por lo que podemos usar estas máscaras para acceder a bits individuales de los números. En particular, el k -ésimo bit de un número es 1 exactamente cuando $x \& (1 \ll k)$ no es cero.

```
1 for (int k = 8; k >= 0; k--) {
2     if (x & (1 << k)) cout << "1";
3     else cout << "0";
4 }
```

Iterar sobre todas las mascasras de tamaño N

```
1 int mask = 0b1011;
2 for (int sub = mask; sub; sub = (sub - 1) & mask) {
3     cout << bitset<4>(sub) << endl; // imprime la submáscara en binario
4 }
```

```

5 // opcional: incluir la máscara cero como submáscara
6 cout << bitset<4>(0) << endl;

```

4.4.4 Propiedades

- Comprobar si un número es par: `x & 1`
- Encender el k -ésimo bit: `x | (1 << k)`
- Apagar el k -ésimo bit: `x & ~(1 << k)`
- Invertir el k -ésimo bit: `x ^ (1 << k)`
- Poner a 0 el último bit en 1: `x & (x - 1)`
- Poner a 0 todos los bits en 1, excepto el último: `x & -x`
- Invertir todos los bits después del último bit en 1: `x | (x - 1)`
- Multiplicar por 2^n : `x << n`
- Dividir por 2^n : `x >> n`
- Comprobar si un número positivo es potencia de dos: `x & (x - 1) == 0`

Código de Gray El código Gray es un sistema numérico en el que los números consecutivos difieren en un solo bit. Cada número en el código Gray se representa en binario, donde un bit cambia su estado entre números consecutivos, lo que facilita la detección y corrección de errores.

Para convertir un número binario a código Gray, si definimos X_2 de N bits como $B_{N-1}, B_{N-2}, B_{N-3}, \dots, B_0$ y su código de Gray como $G_{N-1}, G_{N-2}, G_{N-3}, \dots, G_0$, entonces:

$$G_{N-1} = B_{N-1} \text{ para } n = N-1 \quad G_i = G_{i+1} \oplus B_i \text{ para } i = N-2, N-3, \dots, 0$$

Por ejemplo, si $10_2 = 1010$, donde $B_3 = 1, B_2 = 0, B_1 = 1$ y $B_0 = 0$, entonces: $G_3 = B_3 = 1$ $G_2 = G_3 \oplus B_2 = 1 \oplus 0 = 1$ $G_1 = G_2 \oplus B_1 = 0 \oplus 1 = 1$ $G_0 = G_1 \oplus B_0 = 1 \oplus 0 = 1$ Es decir, convertir el número binario 1010 a código de Gray daría como resultado 1111, y $Gray(10) = 15$.

$$Gray(n) = n \oplus (n \gg 1)$$

```

1 int gray(int n) {
2     return n ^ (n >> 1);
3 }

```

5 Ordenamiento

Por lo regular no es conveniente implementar un método de ordenamiento desde cero, durante una competencia, pero lo que se recomienda usar directamente la STL.

Complejidad: $O(n \log n)$

```

1 sort(all(v)); // sort(v.begin(), v.end())

```

5.1 Operadores de Comparación

La función `sort` requiere de un operador de comparación, que para la gran mayoría de las estructuras STL ya viene implementada, pero esta se puede sobre escribir.

```

1 vector<pair<int, int>> v2 = {{1,2}, {2,3}, {1,5}};
2 sort(all(v2));
3 // [(1,2), (1,5), (2,3)]

```



```

4 vector<tuple<int,int,int>> v3 = {{1,5,3}, {2,1,4}, {2,1,3}};
5 sort(all(v3));
6 // [(1,5,3),(2,1,3),(2,1,4)]

```

Se puede tambien sobrecargar el operador de comparación de la definición de la estructura.

```

1 class Point
2 {
3 public:
4     int x,y;
5     bool operator<(const Point &p)
6     {
7         if (x == p.x) return y < p.y;
8         return x < p.x;
9     }
10 }

```

Tambien se puede pasar una función callback cómo parametro.

```

1 bool comp(string a, string b)
2 {
3     if (a.size() == b.size()) return a < b;
4     return a.size() < b.size();
5 }
6
7 sort(all(v), comp);

```

6 Búsqueda Binaria

Idea: Encontrar un elemento k iterativamente **Complejidad** : $\log n$

```
1 int l = 0, r = n-1;
2 while (l <= r)
3 {
4     int mid = l + (r-l) / 2;
5     if (v[mid] == k) // Found k at index mid
6     else if (v[mid] < k) l = mid+1;
7     else r = mid-1;
8 }
```

Idea: Encontrar un elemento k recursivamente **Complejidad** : $\log n$

```
1 int bs(vi &v, int l, int r, int k)
2 {
3     if (l > r) return -1; // Not found
4     int mid = l + (r-l) / 2;
5     if (v[mid] == k) return mid;
6     else if (v[mid] < k) return bs(v, mid+1, r, k);
7     else return bs(v, l, mid -1, k);
8 }
```

Idea: Encontrar un elemento k con saltos de $\frac{n}{2^i}$ **Complejidad** : $\log n$

```
1 int m = 0;
2 for (int r = n/2; r >=1; r/=2)
3 {
4     while (m+r < n && v[m+r] <= k) m += r;
5 }
6
7 if (v[m] == k) // Found k at index m
```

Idea: Encontrar el valor mas grande que cumpla cierta condición cuando $x < k = false$ y $x \geq true$ **Complejidad** : $\log n$

```
1 int m = -1;
2 int z; // upper_bound that always is true
3 for (int r = z; r >=1 ; r/=2)
4 {
5     while (!valid(m+r)) m += r;
6 }
7 int k = m+1;
```

Ejemplo: Dadas n tablas de longitudes $v[i]$, quieres cortar para obtener al menos k piezas de longitud exacta l

Entrada

```
1 4 11
2 802 743 457 539
```

Salida*

```
1 200
```

```
1 bool valid(vi &v, ll l, ll k)
2 {
3     ll cnt = 0;
4     for (auto x : v) cnt += x / l;
5     return cnt >= k;
6 }
7
8 int bs(vi &v, ll k)
```

```

9 {
10     ll m = 0; // Lower bound
11     ll z = 1e9 + 1; // Upper bound
12     for (ll r = z; r >= 1; r/=2)
13     {
14         while (m+r < z && valid(v, m+r, k))
15             m += r;
16     }
17     return m;
18 }
19
20 void _main()
21 {
22     int n, k;
23     cin>>n>>k;
24     vi v(n);
25     forn(i,0,n) cin>>v[i];
26
27     cout<<bs(v,k)<<endl;
28 }

```

6.1 Búsqueda Binaria con STL

6.1.1 Verificar si existe

```

1 bool exist(vi &v, int k)
2 {
3     return binary_search(all(v), k);
4 }

```

6.1.2 Encontrar índice

```

1 int bs(vi &v, int k)
2 {
3     auto it = lower_bound(all(v), k);
4     if (it != v.end() && *it == k)
5         return distance(v.begin(), it);
6     return -1;
7 }

```

6.1.3 Primer valor que sea menor

```

1 int lowerBound(vi &v, int k)
2 {
3     auto it = lower_bound(all(v), k);
4     if (it == v.begin()) return -1;
5     //it--; //Estrictamente menor
6     return distance(v.begin(), prev(it));
7 }

```

6.1.4 Primer valor que sea mayor

```
1 int upperBound(vi &v, int k)
2 {
3     auto it = upper_bound(all(v), k);
4     if (it == v.end()) return -1;
5     //it--; //Estrictamente mayor
6     return distance(v.begin(), it);
7 }
```

7 Búsqueda Ternaria

Idea: Se usa cuando una función $f(x)$ es **unimodal** (crece y luego decrece, o viceversa). Por ejemplo buscar el máximo en una función discreta. **Complejidad:** $O(\log n)$

```
1 int ts(int l, int r)
2 {
3     int m1, m2, ans;
4     while (r - l >= 3)
5     {
6         m1 = l + (r-l) / 3;
7         m2 = r - (r-l) / 3;
8
9         if (f(m1) < f(m2)) l = m1;
10        else r = m2;
11    }
12 }
```

Busqueda ternaria sobre reales

Idea: Se usa cuando $f(x)$ es continua y unimodal (por ejemplo, funciones convexas o cóncavas).

```
1 double ts(double l, double r)
2 {
3     double m1,m2;
4     forn(i,0,200)
5     {
6         m1 = l + (r-l) / 3;
7         m2 = r - (r-l) / 3;
8
9         if (f(m1) < f(m2)) l = m1;
10        else r = m2;
11    }
12    return (l+r) / 2;
13 }
```

Ejemplo Buscar el valor de x que maximiza $f(x) = -(x-3)^2 + 9$ (máximo en $x = 3$):

```
1 double f(double x)
2 {
3     return -(x-3) * (x-3) + 9;
4 }
5
6 cout<<ts(-100,100)<<endl;
```

8 Combinatoria

8.1 Definición general

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

```
1 ll comb(int n, int k)
2 {
3     return fact(n) / (fact(k) * fact(n-k));
4 }
```

8.2 Fórmula Recursiva

$$\binom{n}{k} = \begin{cases} 1, & \text{si } k = 0 \text{ o } k = n, \\ \binom{n-1}{k-1} + \binom{n-1}{k}, & \text{en otro caso.} \end{cases}$$

```
1 ll comb(int n, int k)
2 {
3     if (k == 0 || k == n) return 1;
4     return comb(n-1, k-1) + comb(n-1, k);
5 }
```

8.3 Obtener todas combianciones

```
1 void combHelper(vi &a, int start, int k, vi &temp, vvi &ans) {
2     if(k == 0) { ans.pb(temp); return; }
3
4     forn(i, start, a.size() - k + 1)
```

```

5     {
6         temp.pb(a[i]);
7         combHelper(arr, i+1, k-1, temp, ans);
8         temp.pop_back();
9     }
10 }
11
12 vvi getCombs(vi &arr, int k) {
13     vvi ans;
14     vi temp;
15     combHelper(arr, 0, k, temp, ans);
16     return ans;
17 }

```

9 Permutaciones

9.1 Definición general

$$P(n, k) = \frac{n!}{(n - k)!}$$

```
1  ll perm(int n, int k) {
2      return fact(n) / fact(n-k);
3  }
```

9.2 Forma recursiva

$$P(n, k) = \begin{cases} 1, & \text{si } k = 0, \\ n \cdot P(n - 1, k - 1), & \text{en otro caso.} \end{cases}$$

```
1  ll perm(int n, int k) {
2      if(k == 0) return 1;
3      else if (n == k) return fact(n);
4      else if (k == 1) return n;
5      return n * perm(n-1, k-1);
6  }
```

9.3 Obtener todas combianciones

```
1  void permHelper(vi &a, int start, vvi &ans) {
2      if(ans.size() == start) { ans.pb(a); return; }
3
4      forn(i, start, a.size() - k)
5      {
6          swap(a[start], a[i]);
7          permHelper(a, start + 1, ans);
8          swap(a[start], a[i]);
9      }
10 }
11
12 vvi getPerms(vi &arr, int k) {
13     vvi ans;
14     permHelper(arr, 0, ans);
15     return ans;
16 }
```

10 Propiedades de Combinatoria y Permutaicones

10.0.1 Teorema Binomial

$$(a+b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$$

```
1 ll binomialTheorem(int a, int b, int n, int k) {  
2     return comb(n, k) * pow(a, n-k) * pow(b, k);  
3 }
```

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

```
1 ll sumComb(int n)  
2 {  
3     return 1LL << n;  
4 }
```

$$P(n, k) = k! \cdot \binom{n}{k}$$

```
1 ll permViaComb(int n, int k) {  
2     return fact(k) * comb(n, k);  
3 }
```

$$P(n+1, k) = (n+1) \cdot P(n, k-1)$$

```
1 ll permRecRelation(int n, int k) {  
2     if(k == 0) return 1;  
3     return (n+1) * permRec(n, k-1);  
4 }
```

10.1 Palo de Hockey

$$\binom{r}{r} + \binom{r+1}{r} + \cdots + \binom{n}{r} = \binom{n+1}{r+1}$$

```
1 ll hockeyStick(int r, int n) {  
2     ll sum = 0;  
3     for(int i = r; i <= n; i++) sum += comb(i, r);  
4     return sum;  
5 }
```

10.2 Números de Catalán

Los números de Catalán se definen como:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

```
1 ll catalan(int n) {  
2     return comb(2*n, n) / (n+1);  
3 }
```


11 Principio de Inclusión - Exclusión

11.1 Dos conjuntos

$$|A \cup B| = |A| + |B| - |A \cap B|$$

```
1 int union2(int a, int b, int ab) {
2     return a + b - ab;
3 }
```

11.2 Tres conjuntos

```
1 int union3(int a, int b, int c, int ab, int ac, int bc, int abc) {
2     return a + b + c - ab - ac - bc + abc;
3 }
```

11.3 Caso general

$$|X_1 \cup X_2 \cup \dots \cup X_n| = \sum_{\emptyset \neq S \subseteq \{1, \dots, n\}} (-1)^{|S|+1} \left| \bigcap_{i \in S} X_i \right|$$

```
1 #include <bitset>
2 #include <vector>
3 using namespace std;
4
5 int inclusionExclusion(int n, vi &sizes)
6 {
7     int ans = 0;
8     for (int m = 1; m < (1 << n); m++)
9     {
10         bitset<32> mask(m);
11         int bits = mask.count();
12         int sz = 0;
13         forn(i, 0, n)
14             if (mask[i]) sz += sizes[i];
15         ans += (bits % 2 ? sz : -sz);
16     }
17     return ans;
18 }
```

11.4 Derangements (Desarreglos)

Número de permutaciones sin elementos en su posición original:

```
1 ll derangements(int n)
2 {
3     if (n == 1) return 0;
4     else if (n == 2) return 1;
5     return (n-1) * (derangements(n-1) + derangements(n-2));
6 }
```

11.5 Burnside's Lemma

Número de combinaciones distintas considerando simetrías, donde $c(k)$ es el número de combinaciones fijas por la k -ésima simetría.

$$\text{distinct} = \frac{1}{n} \sum_{k=0}^{n-1} c(k)$$

```

1 ll burnside(int n, int m)
2 {
3     ll ans = 0;
4     forn(n,0,k) ans += pow(m, gcd(k, n));
5     return ans / n;
6 }

```

12 Exponenciación Binaria

La operación a^b en la gran mayoría de librerías se implementa en $O(b)$. Sin embargo, si representamos b en base 2 (binario), se puede realizar en $O(\log b)$:

```

1 int binpow(int a, int b) {
2     int r = 1;
3     while (b > 0) {
4         if (b & 1) r *= a;
5         a *= a;
6         b >>= 1;
7     }
8     return r;
9 }

```

13 Aritmética Modular

Cuando los números de entrada son muy grandes, los problemas suelen requerir que apliquemos aritmética modular con un número p , típicamente $10^9 + 7$ (1000000007).

Por ejemplo, supongamos que queremos calcular: $r = (a * b * c)$

Si realizamos la multiplicación directamente, los números intermedios pueden exceder la capacidad de almacenamiento, aunque matemáticamente sea correcto. Por eso usamos operaciones modulares paso a paso.

13.0.1 Operaciones mediante aritmética modular

```

1 using u64 = uint64_t;
2 const u64 MOD = 1e9 + 7;
3
4 #define ADD(a, b) (((a \% MOD) + (b \% MOD)) \% MOD)
5 #define SUBTRACT(a, b) (((a) - (b) + MOD)\% MOD)
6 #define MULTIPLY(a, b) (((a) \% MOD) * ((b) \% MOD)) \% MOD
7 #define DIVIDE(a, b) (MULTIPLY(a, binpow(b, MOD-2)))
8
9
10 u64 binpow(u64 a, u64 b) {
11     u64 r = 1;
12     a %= MOD;
13     while (b > 0) {
14         if (b & 1) r = MULTIPLY(r, a);
15         a = MULTIPLY(a, a);
16         b >>= 1;
17     }
18     return r;
19 }
20 u64 modInverse(u64 a) {return binpow(a, MOD - 2);}

```

14 Teoría de Números

14.1 Números Primos y Factores

Definición:

Un número a es **factor** (o **divisor**) de b si a divide a b , denotado $a \mid b$.

Un entero $n > 1$ es **primo** si sus únicos divisores positivos son 1 y n .

Factorización prima única:

Todo entero $n > 1$ se puede escribir de forma única (hasta orden) como:

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$$

14.1.1 Cantidad de factores

Sea $\tau(n)$ el número de factores de un entero n . Por ejemplo, $\tau(12) = 6$, porque los factores de 12 son 1, 2, 3, 4, 6 y 12. Para calcular el valor de $\tau(n)$, podemos usar la fórmula

$$\tau(n) = \prod_{i=1}^k (\alpha_i + 1)$$

ya que para cada primo p_i , hay $\alpha_i + 1$ formas de elegir cuántas veces aparece en el factor. Por ejemplo, como $12 = 2^2 \cdot 3^1$ $\tau(12) = (2 + 1) \cdot 1(2) = 6$.

Complejidad: $O(\sqrt{n})$

```
1  int tau(n)
2  {
3      int ans = 0;
4      for (int i=2; i*i<=n; i++)
5      {
6          int cnt = 0;
7          while (n \% i == 0) { n/=i; cnt++;}
8          ans *= (cnt+1);
9      }
10     if (n > 1) ans *= 2;
11     return ans;
12 }
```

14.2 Suma de factores

Sea $\sigma(n)$ la suma de los factores de un entero n . Por ejemplo, $\sigma(12) = 28$, porque $(1 + 2 + 3 + 4 + 6 + 12 = 28)$. Para calcular el valor de $\sigma(n)$, podemos usar la fórmula

$$\sigma(n) = \prod_{i=1}^k (1 + p_i + \dots + p_i^{\alpha_i}) = \prod_{i=1}^k \frac{p_i^{\alpha_i+1} - 1}{p_i - 1}$$

donde la última forma se basa en la fórmula de progresión geométrica. Por ejemplo, $\sigma(12) = \frac{2^3-1}{2-1} \cdot \frac{3^2-1}{3-1} = 28$

Complejidad: $O(\sqrt{n})$

```
1  ll sigma(n)
2  {
3      ll ans = 0;
4      for (int i=2; i*i<=n; i++)
5      {
6          ll cnt = 0, p = i;
7          while (n % i == 0) { n/=i; cnt++; p *= i;}
8          ans += (cnt+1);
9
10         if(cnt > 0)
11             ans *= (p*i-1) / (i-1);
12     }
13     if (n > 1) ans *= (1LL * n * n - 1) / (i-1);
14     return ans;
15 }
```

14.2.1 Lista de factores

Observa que cada factor primo aparece en el vector tantas veces como divide el número.

Por ejemplo $12 = 2^2 * 3$, por lo que el resultado de la función es $[2, 2, 3]$

Complejidad: $O(\sqrt{n})$

```
1 vi factors(int n) {
2     vi f;
3     for (int x = 2; x*x <= n; x++)
4     {
5         while (n \% x == 0) {f.push_back(x); n /= x;}
6     }
7     if (n > 1) f.push_back(n);
8     return f;
9 }
```

Variante que toma la criba y devuelve un map donde el valor de cada clave es la potencia, ejemplo $12 = \{2 : 2, 3 : 1\}$

Complejidad: $O(\pi(\sqrt{n}) + \log n) \approx O(\frac{\sqrt{n}}{\log n} + \log n)$

```
1 map<ll,ll> factors(ll n, vi primes)
2 {
3     map<ll,ll> f;
4     for (auto d : primes)
5     {
6         if (d*d > n) break;
7         while (n \%d == 0) {f[d]++; n /= d;}
8     }
9     if (n > 1) f[n]++;
10    return f;
11 }
```

14.3 Pruebas de primalidad

Revisar si un número es primo.

```
1 bool isPrime(int n)
2 {
3     if (n < 2) return false;
4     for (int x = 2; x*x <=n; x++)
5         if (n\%x == 0) return false;
6     return true;
7 }
```

14.3.1 Algoritmos probabilísticos

Estos algoritmos se basan en comprobar de manera probabilística basado en el Teorema débil de Fermat, cuanto mayor sea la cantidad de iteraciones, menor es la probabilidad de equivocarse.

Prueba de Fermat La prueba de Fermat puede fallar con los números de Carmichael ($a^{n-1} \equiv 1 \pmod n$ se cumple para todo a coprimo con n , por ejemplo $561 = 3 \times 11 \times 17$), sin embargo en la práctica se sigue usando ya que es bastante rápida y los números de Carmichael son raros.

```
1 bool isPrimeFermat(int n, int iter=5) {
2     if (n < 4) return n == 2 || n == 3;
3     for (int i = 0; i < iter; i++)
4     {
5         int a = 2 + rand() \% (n - 3);
```

```

6         if (binpow(a, n - 1, n) != 1) return false;
7     }
8     return true;
9 }

```

Prueba de Miller Miller demostró que el algoritmo se puede volver determinista si se prueban todas bases $a \leq 2\ln(n)^2$, siendo que para los números de 64 bit, basta con revisar las primeras 12 bases primas.

```

1  using u64 = uint64_t;
2  using u128 = __uint128_t;
3  bool check_composite(u64 n, u64 a, u64 d, int s) {
4      u64 x = binpower(a, d, n);
5      if (x == 1 || x == n - 1) return false;
6      for (int r = 1; r < s; r++)
7      {
8          x = (u128)x * x % n;
9          if (x == n - 1) return false;
10     }
11     return true;
12 }
13 bool MillerRabin(u64 n)
14 {
15     if (n < 2) return false;
16     int r = 0;
17     u64 d = n - 1;
18     while ((d & 1) == 0) { d >>= 1; r++; }
19     for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
20     {
21         if (n == a) return true;
22         if (check_composite(n, a, d, r)) return false;
23     }
24     return true;
25 }

```

14.3.2 Primos como diferencia de cuadrados

Un numero impar se puede escribir $n = pq$ como la diferencia de cuadrados $n = a^2 - b^2$.

$$n = \left(\frac{p+q}{2}\right)^2 - \left(\frac{p-q}{2}\right)^2$$

Estos valores p y q se pueden obtener con el metodo de factorización de Fermat. $O(|p - q|)$ cabe mencionar que si los números son demasiado grande o la distancia entre p y q , este algoritmo es extremadamente lento.

```
1 pair<int,int> fermat(int n) {
2     int a = ceil(sqrt(n));
3     int b2 = a*a - n;
4     int b = round(sqrt(b2));
5     while (b * b != b2)
6     {
7         a = a + 1;
8         b2 = a*a - n;
9         b = round(sqrt(b2));
10    }
11    return {a-b,n/(a-b)};
12 }
```

14.4 Computar números primos

14.4.1 Criba de Erastotenes

Idea: marcar múltiplos de cada primo hasta n .

Complejidad: $O(n \log \log n)$

```
1 vi primes(int n)
2 {
3     vb isPrime(n + 1, true);
4     vi p;
5     forn(i, 2, n+1)
6     {
7         if (isPrime[i])
8         {
9             p.push_back(i);
10            for(11 j = i*i; j <= n; j += i)
11                isPrime[j] = true;
12        }
13    }
14    return p;
15 }
```


14.4.2 Criba Lineal

Idea: Existe una optimización que permite realizar la criba en $O(n)$ a cambio de usar más memoria, que solo debe ser usada si $N \leq 10^7$ **Complejidad:** $O(n)$

```
1  vi linearPrimes(int)
2  {
3      vi mp(n+1); // Minimum Prime Factor
4      vi p;
5
6      forn(i,2,n+1)
7      {
8          if (mp[i] == 0)
9          {
10             mp[i] = i;
11             p.push_back(i);
12          }
13
14          for (int j = 0; i * p[j] <= n; ++j)
15          {
16             mp[i * p[j]] = p[j];
17             if (p[j] == mp[i]) break;
18          }
19      }
20      return p;
21 }
```

14.4.3 Contar número de primos en un rango

Idea: Realizar una suma prefijos **Complejidad de precomputo:** $O(n \log \log \sqrt{n})$ **Complejidad de consulta:** $O(1)$

```
1  vi countPrimes(int n, vb &is_prime)
2  {
3      vi ps(n+1, 0);
4      forn(i,2,n+1)
5      {
6          if (is_prime[i]) ps[i] = ps[i-1] + 1;
7          else ps[i] = ps[i-1];
8      }
9      return ps;
10 }
11
12 int countPrimes(int l, int r, vi &count)
13 {
14     return ((l == 1) ? count[r] : count[r] - count[l-1]);
15 }
```

14.4.4 Encontrar primos en un rango

Encontrar primos en un rango

Encontrar números primos en un rango $[L, R]$ donde $R - L + 1 \approx 10^7$ y $R \leq 10^{12}$.

```
1  vb segmentedSieveNoPreGen(long long vbL, long long r) {
2      vb isPrime(r - 1 + 1, true);
3      ll lim = sqrt(r);
4      for (ll i = 2; i <= lim; ++i)
5          for (ll j = max(i * i, (1 + i - 1) / i * i); j <= r; j += i)
6              isPrime[j - 1] = false;
7
8      if (1 == 1) isPrime[0] = false;
9      return isPrime;
10 }
```

14.5 Algoritmo de Euclides

14.5.1 Máximo Común Divisor

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Complejidad: $O(\log \min(a, b))$.

```
1  int gcd(int a, int b)
2  {
3      return b == 0 ? a : gcd(b, a % b);
4  }
```

14.5.2 Máximo Común Multiplo

$$\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

Complejidad: $O(\log \min(a, b))$.

```
1  int lcm(int a, int b)
2  {
3      return a / gcd(a, b) * b;
4  }
```

Euclides Extendido

Devuelve x, y tales que $ax + by = \gcd(a, b)$ — útil para inversos modulares y diofánticas.

```
1 int egcd(int a, int b, int *x, int *y)
2 {
3     if (b == 0) { *x = 1; *y = 0; return a; }
4     int x1, y1;
5     int g = egcd(b, a % b, &x1, &y1);
6     *x = y1;
7     *y = x1 - (a / b) * y1;
8     return g;
9 }
```

14.5.3 Numeros Coprimos

Dos enteros a y b son coprimos si $\gcd(a, b) = 1$.

```
1 bool coprimes(int a, int b)
2 {
3     return gcd(a, b) == 1;
4 }
```

14.5.4 Funcion Totiente de Euler

La función totiente de Euler $\varphi(n)$ da el número de enteros entre 1 y n que son coprimos con n si

$$n = p_1^{\alpha_1} \cdots p_k^{\alpha_k},$$

$$\varphi(n) = \prod_{i=1}^k p_i^{\alpha_i-1} (p_i - 1)$$

Ejemplo: $10 = 2 \cdot 5 \Rightarrow \varphi(10) = 2^{1-1}(2-1) \cdot 5^{1-1}(5-1) = 4$

Complejidad: $O(\sqrt{n})$

```
1 ll phi(ll n) {
2     ll result = n;
3     for (ll p = 2; p * p <= n; ++p) {
4         if (n % p == 0) {
5             while (n % p == 0) n /= p;
6             result -= result / p;
7         }
8     }
9     if (n > 1) result -= result / n;
10    return result;
11 }
```

14.5.5 Teorema de Euler

Teorema de Euler:

$$x^{\varphi(m)} \equiv 1 \pmod{m}, \quad x \text{ coprimo con } m$$

Si m es primo ($\varphi(m) = m - 1$):

14.5.6 Pequeño Teorema de Fermat

$$x^{m-1} \equiv 1 \pmod{m}$$

14.6 Inversos Multiplicativos Modulares

El inverso multiplicativo de x módulo m , $\text{inv}_m(x)$, cumple:

$$x \cdot \text{inv}_m(x) \equiv 1 \pmod{m}$$

Existe si y solo si $\text{gcd}(x, m) = 1$, y se puede calcular con:

$$\text{inv}_m(x) = x^{\varphi(m)-1} \pmod{m}$$

Si m es primo:

$$\text{inv}_m(x) = x^{m-2} \pmod{m}$$

Ejemplo: $\text{inv}_{17}(6) = 6^{15} \bmod 17 = 3$

```
1  u64 binpow(u64 a, u64 b) {
2      u64 r = 1;
3      a %= MOD;
4      while (b > 0) {
5          if (b & 1) r = MULTIPLY(r, a);
6          a = MULTIPLY(a, a);
7          b >>= 1;
8      }
9      return r;
10 }
11 u64 modInverse(u64 a) {return binpow(a, MOD - 2);}
```

14.7 Ecuaciones Diofánticas

Una ecuación diofántica tiene la forma:

$$ax + by = c, \quad a, b, c \in \mathbb{Z}$$

Se puede resolver si c es divisible por $\gcd(a, b)$.

Si (x_0, y_0) es una solución, todas las soluciones son:

$$\begin{cases} x = x_0 + k \frac{b}{\gcd(a, b)} \\ y = y_0 - k \frac{a}{\gcd(a, b)} \\ k \in \mathbb{Z} \end{cases}$$

Ejemplo: resolver $39x + 15y = 12$ $\gcd(39, 15) = 3 \mid 12$

Usando el algoritmo de Euclides extendido:

$$39 \cdot 2 + 15 \cdot (-5) = 3 \implies 39 \cdot 8 + 15 \cdot (-20) = 12$$

```
1 pair<bool, pair<ll, ll>> solve_diophantine(ll a, ll b, ll c) {
2     ll x0, y0;
3     ll g = ext_gcd(llabs(a), llabs(b), x0, y0);
4     if (c % g != 0) return {false, {0, 0}};
5     if (a < 0) x0 = -x0;
6     if (b < 0) y0 = -y0;
7     ll mult = c / g;
8     x0 *= mult;
9     y0 *= mult;
10    return {true, {x0, y0}};
11 }
```

14.7.1 Teorema Chino del Resto

Sistema de congruencias:

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_n \pmod{m_n} \end{cases}$$

con m_i coprimos dos a dos.

Solución general:

$$x = \sum_{k=1}^n a_k X_k \operatorname{inv}_{m_k}(X_k)$$

donde

$$X_k = \frac{m_1 m_2 \cdots m_n}{m_k}$$

Ejemplo:

$$\begin{cases} x \equiv 3 \pmod{5} \\ x \equiv 4 \pmod{7} \\ x \equiv 2 \pmod{3} \end{cases} \implies x = 263$$

Infinitas soluciones:

$$x + m_1 m_2 \cdots m_n$$

```
1 struct Congruence{
2     ll a, m;
3 };
4
5 ll chinese_remainder_theorem(vector<Congruence> & cgrs)
6 {
7     ll m = 1, ans = 0, ai, mi, ni;
8     for (auto &c : cgrs) m *= c.m;
9
10    for (auto const& c : cgrs) {
11        ai = c.a;
12        mi = m / c.m;
13        Ni = mod_inv(mi, c.m);
14        ans = (ans + ai * mi \% m * ni) \% m;
15    }
16    return ans;
17 }
```

14.8 MEX

El **MEX** (“minimum excludant”) de un conjunto de números enteros es el **mínimo número entero no presente en el conjunto**.

Ejemplo: $\text{mex}(\{0, 1, 3\}) = 2$

```
1 int mex(vi &v)
2 {
3     set<int> s(all(v));
4     int m = 0;
5     while (s.count(m)) m++;
6     return m;
7 }
```

15 Juegos NIM

- Hay (n) heaps con (x_i) palillos.
- Jugadores alternan turnos.
- En cada turno se remueven palillos de un heap no vacío.
- Gana quien remueve el último palillo.

15.1 Nim Sum

$$s = x_1 \oplus x_2 \oplus \cdots \oplus x_n$$

- Si $s = 0$ estado perdedor.
- Si $s \neq 0$ estado ganador.

15.2 Movimiento óptimo

Para un estado ganador:

- Elegir heap i tal que $x_i \oplus s < x_i$
- Remover hasta que el heap tenga $x_i \oplus s$ palillos.

15.3 Teorema de Sprague–Grundy

El Teorema de Sprague–Grundy generaliza la estrategia de Nim a cualquier juego que cumpla:

- Dos jugadores alternan turnos.
- El juego consiste en estados, y los movimientos posibles dependen solo del estado.
- El juego termina cuando un jugador no puede moverse.
- El juego termina seguro eventualmente.
- Los jugadores tienen información completa y no hay aleatoriedad.

15.4 Números de Grundy

Cada estado del juego se asocia con un **número de Grundy** ($G(s)$), equivalente al tamaño de un heap de Nim.

Si s tiene movimientos hacia los estados $s_1, s_2, \dots, s_n$:

$$\text{mex}\{g_1, g_2, \dots, g_n\}$$

16 Estructuras de Datos

16.1 Arreglos Dinámicos

Un arreglo dinámico es un arreglo cuyo tamaño puede cambiar durante la ejecución del programa.

La biblioteca estándar de C++ (STL) proporciona varias estructuras de arreglos dinámicos, siendo la más útil el vector.

16.1.1 Vectores

Un vector permite agregar y eliminar elementos eficientemente al final de la estructura.

Inicialización de un vector vacío

```
1  vi v;  
2  v.pb(3);  
3  v.pb(2);  
4  v.pb(5);
```

Inicialización de un vector con lista de elementos

```
1  vi a = {2, 4, 2, 5, 1};
```

Inicialización de un vector determinando el tamaño y valores iniciales

```
1  vi a(8); // tamaño 8, valor inicial 0  
2  vi b(8,2); // tamaño 8, valor inicial 2
```

Los elementos pueden ser accedidos de manera ordinaria

```
1  cout << v[0] << "\n"; // 3  
2  cout << v[1] << "\n"; // 2  
3  cout << v[2] << "\n"; // 5
```


La función `size()` retorna el número de elementos en el vector

```
1     forn(i,0,v.size())
2         cout << v[i] << " ";
3     cout << endl;
```

Manera más corta de iterar a través de un vector

```
1     for (auto x : v) {
2         cout <<x<< "\n";
3     }
```

La función `back()` retorna el último elemento de un vector, y la función `pop_back()` elimina el último elemento

```
1     vi v = {2,4,2,5,1};
2     cout << v.back() << "\n"; // 1
3     v.pop_back();
4     cout << v.back() << "\n"; // 5
```

Los vectores se implementan de manera que las operaciones `push_back()` y `pop_back()` trabajen en $O(1)$ tiempo medio. En la práctica, el uso de un vector es casi tan rápido como el uso una arreglo ordinario.

16.1.2 Iteradores y rangos

Los iteradores son objetos que permiten recorrer los elementos de una estructura de datos de forma genérica. Funcionan como punteros que apuntan a elementos dentro de contenedores de la STL, como vector, set o map.

Iterador Es una variable que apunta a un elemento de una estructura de datos.

- El iterador `begin()` apunta al primer elemento de una estructura de datos.
- El iterador `end()` apunta a la posición después del último elemento

Los iteradores permiten recorrer estructuras sin conocer su implementación interna.

El elemento al que apunta un iterador se puede acceder con la sintaxis `*`.

```
1     cout << *v.begin() << "\n"; //primer elemento del vector
```

Rango Es una secuencia de elementos consecutivos en una estructura de datos.

Los iteradores `begin()` y `end()` definen un rango que contiene todos los elementos de una estructura de datos.

El código siguiente ordena primero un vector, luego invierte el orden de sus elementos, y por último, mezcla sus elementos.

```
1     sort(v.begin(),v.end());
2     reverse(v.begin(),v.end());
3     random_shuffle(v.begin(),v.end());
```

Un ejemplo más útil, `lower_bound` da un iterador al primer elemento en un rango ordenado cuyo valor es al menos x, y `upper_bound` da una iterador al primer elemento cuyo valor es mayor que x

```
1     vi v = {2,3,3,5,7,8,8,8};
2     auto a = lower_bound(v.begin(),v.end(),5);
3     auto b = upper_bound(v.begin(),v.end(),5);
4     cout << *a << " " << *b << "\n"; // 5 7
```

El rango dado debe estar ordenado.

16.1.3 Deque

Un deque es un arreglo dinámico que puede manipularse eficientemente en ambos extremos.

Los deques deben utilizarse si hay necesidad de manipular ambos extremos del arreglo.

```
1 deque<int> d;  
2 d.push_back(5); // [5]  
3 d.push_back(2); // [5,2]  
4 d.push_front(3); // [3,5,2]  
5 d.pop_back(); // [3,5]  
6 d.pop_front(); // [5]
```

16.1.4 Stack

Una pila tiene las funciones push y pop para insertar y quitar elementos al final de la estructura, y la función top que recupera el último elemento

```
1 stack<int> s;  
2 s.push(2); // [2]  
3 s.push(5); // [2,5]  
4 cout << s.top() << "\n"; // 5  
5 s.pop(); // [2]  
6 cout << s.top() << "\n"; // 2
```

16.1.5 Queue

En una cola, los elementos se insertan al final de la estructura y se eliminan desde la parte delantera de la estructura. Ambas funciones front y back permiten acceder al primer y último elemento.

```
1 queue<int> q;  
2 q.push(2); // [2]  
3 q.push(5); // [2,5]  
4 cout << q.front() << "\n"; // 2  
5 q.pop(); // [5]  
6 cout << q.back() << "\n"; // 5
```

16.2 Estructuras Set

16.2.1 Sets y multisets

Set Se basa en un árbol de búsqueda binario equilibrado y sus operaciones funcionan en $O(\log n)$ tiempo.

Unordered set Se basa en una tabla hash y sus operaciones funcionan, por término medio, en $O(1)$.

Puesto que son utilizados de la misma manera, nos centramos en la estructura del set en los siguientes ejemplos.

La función *insert* añade un elemento al set, la función *count* devuelve el número de ocurrencias de un elemento en el set, y la función *erase* elimina un elemento del set.

```
1 set<int> s;  
2 s.insert(3);  
3 s.insert(2);  
4 s.insert(5);  
5 cout << s.count(3) << "\n"; // 1  
6 cout << s.count(4) << "\n"; // 0  
7 s.erase(3);  
8 s.insert(4);  
9 cout << s.count(3) << "\n"; // 0
```

```
10 cout << s.count(4) << "\n"; // 1
```

Una propiedad importante de los sets es que todos sus elementos son distintos, por lo que la función `insert` nunca agregará un elemento si ya se encuentra dentro del set.

La función `count` siempre devuelve 0 (si el elemento no está en el conjunto) o 1 (si está presente)

```
1 set<int> s;  
2 s.insert(3);  
3 s.insert(3);  
4 s.insert(3);  
5 cout << s.count(3) << "\n"; // 1
```

```
1 cout << s.size() << "\n"; // imprimir número de elementos  
2 for (auto x : s) { // recorrer elementos del set  
3     cout << x << "\n";  
4 }
```

La función `find(x)` devuelve un iterador que apunta al elemento cuyo valor es x. Si el conjunto no contiene x, el iterador devuelto será `end()`

```
1 auto it = s.find(x);  
2 if (it == s.end()) {  
3     // x no se encuentra  
4 }
```

Sets ordenados La principal diferencia entre las dos estructuras es que `set` es ordenado, mientras que `unordered_set` no lo es.

```
1 auto first = s.begin();
2 auto last = s.end(); last--;
3 cout << *first << " " << *last << "\n";
```

```
1 cout << *s.lower_bound(x) << "\n";
2 cout << *s.upper_bound(x) << "\n";
```

Multisets Un multiset es un conjunto que puede tener varias copias del mismo valor. C++ incluye las estructuras `multiset` y `unordered_multiset`, que son análogas a `set` y `unordered_set`.

El siguiente código agrega tres copias del valor 5 a un multiset

```
1 multiset<int> s;
2 s.insert(5);
3 s.insert(5);
4 s.insert(5);
5 cout << s.count(5) << "\n"; // 3
```

La función `erase` elimina todas las copias de un valor en un multiset

```
1 s.erase(5);
2 cout << s.count(5) << "\n"; // 0
```

Para eliminar una sola copia de un valor, se puede hacer de la siguiente manera

```
1 s.erase(s.find(5));
2 cout << s.count(5) << "\n"; // 2
```

16.2.2 Maps

Un map es un conjunto que consiste en pares clave-valor (key-value pairs). Las claves pueden ser de cualquier tipo de dato y no tienen que ser consecutivas.

La biblioteca estándar de C++ contiene dos estructuras de map:

- `map` se basa en un árbol binario de búsqueda balanceado, y el acceso a los elementos toma tiempo $O(\log n)$.
- `unordered_map` utiliza hashing, y el acceso a los elementos toma tiempo $O(1)$ en promedio

```
1 map<string, int> m;
2 m["monkey"] = 4;
3 m["banana"] = 3;
4 m["harpsichord"] = 9;
5 cout << m["banana"] << "\n"; // 3
```

El siguiente código recorre e imprime todos los pares clave-valor almacenados:

```
1 for (auto x : m) {
2     cout << x.first << " " << x.second << "\n";
3 }
```

Los elementos de un map están ordenados por sus claves, mientras que los de un `unordered_map` no lo están.

Eliminar una clave de un mapa

```
1 m.erase("banana");
```

Comprobar si una clave existe utilizando la función `count`, que devuelve 1 si la clave está presente, o 0 en caso contrario.

```

1  if (m.count("banana")) {
2      cout << "banana existe\n";
3  }

```

16.2.3 Priority Queues (Colas de Prioridad)

Una cola de prioridad es una estructura similar a un multiset, que permite insertar elementos y obtener/eliminar rápidamente el mínimo o máximo elemento, dependiendo del tipo de cola.

Por defecto, está implementada como un heap binario máximo, lo que permite obtener y eliminar el mayor elemento en la estructura.

```

1  priority_queue<int> q;
2  q.push(3);
3  q.push(5);
4  q.push(7);
5  q.push(2);
6
7  cout << q.top() << "\n"; // 7
8  q.pop();
9  cout << q.top() << "\n"; // 5
10 q.pop();
11
12 q.push(6);
13 cout << q.top() << "\n"; // 6
14 q.pop();

```

Si se desea que la cola devuelva el menor elemento (un min-heap), se puede usar un comparador:

```

1  priority_queue<int, vector<int>, greater<int>> q;

```

16.2.4 Policy-Based Sets (Conjuntos Basados en Políticas)

El compilador g++ ofrece estructuras adicionales que no forman parte de la biblioteca estándar de C++, conocidas como policy-based data structures. Para utilizarlas se deben incluir las siguientes líneas:

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
```

Definición para valores int:

```
1 typedef tree<int, null_type, less<int>, rb_tree_tag,
2 tree_order_statistics_node_update> indexed_set;
```

Ejemplo de uso

```
1 indexed_set s;
2 s.insert(2);
3 s.insert(3);
4 s.insert(7);
5 s.insert(9);
```

Su ventaja es que permite acceder a los elementos por su posición ordenada.

- `find_by_order(k)` devuelve un iterador al elemento en la posición k (0-indexado).
- `order_of_key(x)` devuelve cuántos elementos son menores que x.

```
1 auto x = s.find_by_order(2);
2 cout << *x << "\n"; // 7
3 cout << s.order_of_key(7) << "\n"; // 2
```

17 Monotonic Stack

Es una pila que mantiene sus elementos en un orden creciente o decreciente de forma continua. Al insertar un nuevo elemento, se eliminan los elementos que violan el orden monótono para mantener la consistencia.

Se recorre el arreglo de izquierda a derecha manteniendo una pila de elementos. En cada paso: 1. Mientras el elemento del tope de la pila sea mayor o igual que el actual, se elimina. 2. Si la pila no está vacía, el elemento del tope es el más cercano menor. 3. Luego se agrega el elemento actual a la pila.

Ejemplo de implementación

```
1 vi a = {1, 3, 4, 2, 5, 3, 4, 2};
2 stack<int> s;
3
4 forn(i, 0, sz(a))
5 {
6     while (!s.empty() && s.top() >= a[i])
7         s.pop();
8
9     if (s.empty()) cout << "_ ";
10    else cout << s.top() << " ";
11
12    s.push(a[i]);
13 }
14 cout << endl;
```

Este patrón puede modificarse para hallar:

- El siguiente mayor: pila monótona decreciente
- El siguiente menor: pila monótona creciente

18 Programación Dinámica

Identificar el subproblema

- Encuentra la **pequeña versión del problema** que puedes resolver y que se reutiliza.
- Pregúntate: “Si conozco la solución a los casos más pequeños, ¿puedo construir la solución al caso grande?”
- Ejemplo: En Coin Change, subproblema = “mínimo número de monedas para sumar x”.

Formular la recurrencia

- Escribe cómo se relaciona un caso con los casos más pequeños.
- Generalmente tiene la forma:

$$dp[x] = \text{función de } dp[\text{casos más pequeños}]$$

- Ejemplo:

$$\text{solve}(x) = \min_{c \in \text{coins}} (\text{solve}(x - c) + 1)$$

Definir casos base

- Piensa en **valores triviales** que no dependen de otros subproblemas.
- Ejemplo Coin Change:

- `solve(0) = 0` → no necesitas monedas para sumar 0
- `solve(x < 0) = ∞` → imposible

Memoización o tabla iterativa

- **Memoización:** Guardar resultados de subproblemas recursivos en un array.
- **Iterativa (bottom-up):** Llenar un array desde los casos base hasta el objetivo.
- Ventaja iterativa: menos sobrecarga de llamadas recursivas y más control sobre complejidad.

Reconstrucción de la solución

- A veces necesitamos **no solo el valor óptimo**, sino **cómo construirlo**.
- Guardar información adicional en arrays auxiliares, por ejemplo:
 - `first[x]` → primer paso para llegar a x
- Luego se recorre hacia atrás para reconstruir la solución.

Contar soluciones

- A veces queremos **todas las formas posibles** de alcanzar un estado.
- La recurrencia cambia de “min” o “max” a “suma”:

$$\text{count}[x] = \sum_{c \in \text{coins}} \text{count}[x - c]$$

Optimización de espacio

- Muchos DP pueden reducir el tamaño del array si solo dependen de los últimos k valores.
- Ejemplo: Coin Change solo depende de `dp[x-c]` → se puede usar un vector de tamaño n+1.

18.0.1 Patrones Tipicos

18.0.2 Lineal (1D DP)

Subproblema depende de un solo parámetro, ej. suma o longitud. Ejemplo: Coin Change (mínimo número de monedas para sumar x)

Recursivo:

$$\text{solve}(x) = \begin{cases} \infty & x < 0 \\ 0 & x = 0 \\ \min_{c \in \text{coins}} (\text{solve}(x - c) + 1) & x > 0 \end{cases}$$

```
1 #define INF 1e9
2 vi value(n+1, INF);
3 value[0] = 0;
4 forn (x, 1, n+1) {
5     for (auto c : coins) if (x-c >= 0)
6         value[x] = min(value[x], value[x-c]+1);
7 }
```

18.1 Conjuntos (Bitmask DP)

Ejemplo: Traveling Salesman Problem (TSP) – recorrer todos los nodos con mínimo costo. **Idea:** Representamos cada subconjunto con un bitmask; $\text{dp}[\text{mask}][i]$ = mínimo costo para llegar a nodo i visitando los nodos de mask .

Recursivo:

$$\text{dp}[\text{mask}][i] = \min_{j \in \text{mask}, j \neq i} (\text{dp}[\text{mask} \setminus \{i\}][j] + \text{cost}[j][i])$$

```
1 int dp[1<<n][n];
2 forn (int mask=0; mask<(1<<n); mask++)
3     forn (i, 0, n)
4         dp[mask][i] = INF;
5
6 dp[1][0] = 0; // start at node 0
7 forn (int mask=1; mask<(1<<n); mask++)
8 {
9     forn (u, 0, n)
10    {
11        if (mask & (1<<u))
12        {
13            forn (v, 0, n)
14            {
15                if (mask & (1<<v) && u!=v)
16                    dp[mask][u] = min(dp[mask][u], dp[mask^(1<<u)][v]+cost[v][u]);
17            }
18        }
19    }
20 }
```


18.2 Intervalos

Subproblema depende de un rango $[i, j]$. **Ejemplo:** Matrix Chain Multiplication – minimizar costo de multiplicación de matrices. **Idea:** Para cada intervalo $[i, j]$, probamos todos los puntos k donde podemos dividir el problema en dos subintervalos.

Recursivo:

$$dp[i][j] = \begin{cases} 0 & i = j \\ \min_{k=i}^{j-1} (dp[i][k] + dp[k+1][j] + cost[i][k][j]) & i < j \end{cases}$$

```
1 vvi dp(n, vi(n, 0));
2 forn (len, 2, n+1)
3 {
4     for(int i=0; i<=n-len; i++)
5     {
6         int j=i+len-1;
7         dp[i][j] = INF;
8         forn(k, i, j)
9             dp[i][j] = min(dp[i][j], dp[i][k]+dp[k+1][j]+cost(i,k,j));
10    }
11 }
```

18.3 Longest Increasing Subsequence (LIS)

- Encontrar la subsecuencia creciente más larga en un arreglo.
- Subproblema: $length(k)$ = largo de la LIS que termina en k .

Recursivo:

$$length(k) = \begin{cases} 1 & \text{si no existe } i < k \text{ con } array[i] < array[k] \\ \max_{i < k, array[i] < array[k]} (length(i) + 1) & \text{en otro caso} \end{cases}$$

```
1 vi len(n, 1);
2 forn (k, 0, n)
3 {
4     forn(i, 0, k)
5     {
6         if(v[i]<v[k])
7             len[k] = max(len[k], len[i]+1);
8     }
9 }
10 int LIS = *max_element(all(len));
```

18.4 Paths in a Grid

- Encontrar la ruta de suma máxima desde la esquina superior izquierda a la inferior derecha, solo movimientos hacia abajo o derecha.
- Subproblema: $\text{sum}(y, x)$ = suma máxima hasta la celda (y, x) .

Recursivo:

$$\text{sum}(y, x) = \max(\text{sum}(y - 1, x), \text{sum}(y, x - 1)) + \text{value}[y][x]$$

```
1  forn(y, 1, n+1)
2  {
3      for(x, 1, n+1)
4          sum[y][x] = max(sum[y-1][x], sum[y][x-1]) + value[y][x];
5  }
```

18.5 Knapsack

- Problema: Dado un conjunto de objetos con pesos $w[i]$ y valores $v[i]$, encontrar el subconjunto de máximo valor sin superar capacidad c .
- Subproblema: $dp[i][x]$ = máximo valor usando los primeros i objetos con capacidad x .

Recursivo:

$$dp[i][x] = \begin{cases} dp[i-1][x] & \text{si no usamos el objeto } i \\ dp[i-1][x - w[i]] + v[i] & \end{cases}$$

```
1  struct Item
2  {
3      int val, w;
4  };
5
6
7  pair<vvi, vvb> knapsack(vector<Item>&v, int k)
8  {
9      int n = v.size();
10     vvi dp(n+1, vi(k+1, 0));
11     vvb take(n+1, vb(k+1, false));
12
13     forn(i, 1, n+1)
14     {
15         forn(c, 0, k+1)
16         {
17             dp[i][c] = dp[i-1][c]; // No tomar
18             if (c >= v[i-1].w &&
19                 dp[i-1][c - v[i-1].w] + v[i-1].val > dp[i][c])
20             {
21                 dp[i][c] = dp[i-1][c - v[i-1].w].val + v[i-1].val;
22                 take[i][c] = true;
23             }
24         }
25     }
26     return {dp, take};
27 }
```

18.5.1 Reconstruir Knapsack

```
1  vi rebuild(vector<Item>items, vvb &take, int k)
2  {
3      int n = items.size();
4      vi select;
5      for(int i = n; i >=1; i--)
6      {
7          if (take[i][k])
8          {
9              select.push_back(i-1);
10             k -= items[i-1].w;
11         }
12     }
13     reverse(all(select));
14     return select;
15 }
```

Ejemplo

```
1  vector<Item>items;
2  int capacity = 7;
3
4  auto [dp, take] = knapsack(items, take, capacity);
5  cout << "Max: " << dp[dp.size()-1][capacity] << endl;
6
7
8  vi selected = rebuild(items, take, capacity);
9  for (auto idx : selected) cout << idx << endl;
```

Elevator / Subset DP

- Mínimo número de viajes para subir personas con límite de peso.
- Subproblema: Para cada subconjunto S , calcular $(rides(S), last(S))$.

Código:

```
1 struct Item
2 {
3     int val, w;
4 };
5
6 pair<int, int> best[1<<N];
7
8 void solve_knapsack_mask(vector<Item>&v, int k)
9 {
10     int n = v.size();
11     best[0] = {1, 0}; // subconjunto vacío
12     forn(s, 1, (1<<n))
13     {
14         best[s] = {n+1, 0};
15         forn(p, 0, n)
16         {
17             if (s & (1<<p))
18             {
19                 auto op = best[s ^ (1<<p)];
20                 if (op.second + v[p].w <= k)
21                     op.second += v[p].w;
22                 else
23                 {
24                     op.first++;
25                     op.second = v[p].w;
26                 }
27                 best[s] = min(best[s], op);
28             }
29         }
30     }
31 }
```

18.6 Counting Tilings (Grid DP)

- Contar formas de llenar un grid $n \times m$ con piezas 1×2 y 2×1 .
- Subproblema: $\text{count}(k, x)$ = formas de llenar filas $1..k$ con estado x en fila k .

Idea:

- Representar cada fila como un estado.
- $\text{count}(k, x)$ depende solo de $\text{count}(k-1, y)$ donde y es compatible con x .
- Optimización: representar solo columnas con parte superior de los tiles verticales $\rightarrow 2^m$ estados.

Código:

```
1 vi prev(1<=m,0), curr(1<=m,0);
2 prev[0] = 1;
3 for(row, 1, n+1)
4 {
5     for(int mask=0; mask<(1<=m); mask++)
6     {
7         for(int pmask=0; pmask<(1<=m); pmask++)
8         {
9             if(compatible(mask, pmask))
10                curr[mask] += prev[psmask];
11         }
12     }
13     swap(prev, curr);
14     fill(all(curr), 0);
15 }
```

19 Grafos

19.1 Terminología

Grafo:

Conjunto de nodos (vértices) conectados por aristas.

La convención de nombres es n = número de nodos y m = número de aristas.

Camino:

Secuencia de nodos conectados por aristas. La longitud del camino es el número de aristas que lo componen.

Ciclo:

Camino que inicia y termina en el mismo nodo, por ejemplo $1 \rightarrow 3 \rightarrow 4 \rightarrow 1$.

Grafo conexo:

Existe un camino entre cualquier par de nodos. Si no lo hay, el grafo es desconexo.

Componentes:

Partes conexas e independientes de un grafo, por ejemplo $\{1, 2, 3\}$, $\{4, 5, 6, 7\}$, $\{8\}$.

Árbol:

Grafo conexo que no contiene ciclos, por ejemplo uno con 5 nodos y 4 aristas.

Grafo dirigido:

Las aristas tienen una dirección ($u \rightarrow v$), por ejemplo $3 \rightarrow 1 \rightarrow 2 \rightarrow 5$.

Grafo ponderado:

Cada arista tiene un peso o longitud, por ejemplo $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ con pesos $1 + 7 + 3 = 11$.

Vecinos o adyacentes:

Nodos conectados directamente por una arista, por ejemplo los vecinos de 2 son $\{1, 4, 5\}$.

Grado de un nodo:

Número de vecinos de un nodo, por ejemplo $\text{deg}(2) = 3$.

Suma de grados:

La suma de los grados en un grafo siempre es par: $\sum \deg(v) = 2m$.

Grafo regular:

Todos los nodos tienen el mismo grado d , por ejemplo $d = 3$ para todo nodo.

Grafo completo:

Cada par de nodos está conectado por una arista, de modo que $\deg(v) = n - 1$.

Grado de entrada (indegree):

Número de aristas que llegan a un nodo, por ejemplo $\text{indegree}(2) = 2$.

Grado de salida (outdegree):

Número de aristas que salen de un nodo, por ejemplo $\text{outdegree}(2) = 1$.

Grafo bipartito:

Sus nodos pueden colorearse con dos colores sin que adyacentes compartan color, por ejemplo $A = \{1, 3, 5\}$ y $B = \{2, 4, 6\}$.

Propiedad:

Un grafo es bipartito si y sólo si no tiene ciclos impares, es decir, bipartito $\Leftrightarrow$ sin ciclos impares.

19.2 Representaciones de Grafos

19.2.1 Lista de Adyacencia

Grafos No Ponderados En esta representación, cada nodo x tiene una lista con los nodos a los que está conectado mediante una arista. Para grafos no dirigidos se puede guardar de la misma manera pero cada arista se guarda en ambos sentidos.

```
1 vvi graph(n);
2
3 graph[1].push_back(2);
4 graph[2].push_back(3);
5 graph[2].push_back(4);
6 graph[3].push_back(4);
7 graph[4].push_back(1);
```

Grafos Ponderados En este caso cada lista de nodos u contiene pares de v, w siempre que exista un camino entre u y v con peso w .

```
1 using pi = pair<int,int>;
2 using vpi = vector<pi>;
3 using vvpi = vector<vpi>;
4
5 vvpi graph(n);
6
7 graph[1].push_back({2,5});
8 graph[2].push_back({3,7});
9 graph[2].push_back({4,6});
10 graph[3].push_back({4,5});
11 graph[4].push_back({1,2});
```

Con la lista de adyacencia nos permite de manera eficiente iterar sobre todos los nodos que están conectados a un cierto nodo.

Para grafos no ponderados

```
1 for (auto v : graph[u])
2 {
3     // Process node v
4 }
```

Para grafos ponderados

```

1 for (auto &c : graph[u])
2 {
3     int v = c.first;
4     int w = c.second;
5
6     // Process node v
7 }

```

19.2.2 Matiz de Adyacencia

19.2.3 Lista de Aristas

```

1 using tp = tuple<int,int,int>;
2 using vtp = vector<tp>;
3 using vvtp = vector<vtp>;
4
5 vvtp edges;
6 edges.push_back({1,2,5});

```

19.3 Recorridos de Grafos

19.3.1 Depth First Search (DFS)

La búsqueda en profundidad (DFS) es una técnica simple para recorrer grafos.

El algoritmo empieza en un nodo inicial y avanza hacia todos los nodos alcanzables desde él, siguiendo las aristas del grafo.

Recursivo

```

1 void dfs(int u, vvi &graph, vb &visited)
2 {
3     if (visited) return;
4     visited[u] = true;
5
6     // process node u
7     for (auto v : graph[u])
8         dfs(v, graph, visited);
9 }

```

Iterativo

```

1 void dfs(int s, vvi &graph)
2 {
3     vb visited(graph.size(), false);
4     stack<int> st;
5
6     visited[s] = true;
7     st.push(s);
8
9     while (!st.empty())
10    {
11        int u = st.top(); st.pop();
12        // process node u
13        for (auto v : graph[u])
14        {
15            if (visited[v]) continue;
16            visited[v] = true;
17            st.push(v);
18        }
19    }

```

```
19     }
20
21 }
```

19.4 Breadth First Search (BFS)

La búsqueda en anchura (BFS) recorre los nodos de un grafo en orden creciente de distancia desde el nodo inicial.

De esta forma, se puede calcular la distancia del nodo inicial a todos los demás.

```
1  void dfs(int s, vvi &graph)
2  {
3      int n = graph.size();
4      vb visited(n, false);
5      vi distance(n, -1);
6      queue<int> q;
7
8      visited[s] = true;
9      distance[s] = 0;
10     q.push(s);
11
12     while (!q.empty())
13     {
14         int u = st.front(); st.pop();
15         // process node u
16         for (auto v : graph[u])
17         {
18             if (visited[v]) continue;
19             visited[v] = true;
20             distance[v] = distance[u] + 1;
21             q.push(v);
22         }
23     }
24
25 }
```


19.5 Comprobaciones

19.5.1 Conectividad

Un grafo es conexo si existe un camino entre cualquier par de nodos. Para comprobarlo, basta iniciar desde un nodo y verificar si se puede llegar a todos los demás.

```
1  bool connected(vvi &graph, int s)
2  {
3      vb visited(graph.size(), false);
4      queue<int>q;
5
6      q.push(s);
7      visited[s] = true;
8
9      while (!q.empty())
10     {
11         int u = q.front(); q.pop();
12         for (auto v : graph[u])
13             {
14                 if (!visited[v])
15                     {
16                         visited[v] = true;
17                         q.push(v);
18                     }
19             }
20     }
21
22     for(auto x : visited)
23         if (!x) return false;
24     return true ;
25 }
```

19.5.2 Ciclos

Un grafo tiene un ciclo si, al recorrerlo, encontramos un nodo cuyo vecino (distinto del anterior en el camino actual) ya fue visitado.

```
1  bool hasCycle(vvi &graph)
2  {
3      int n = graph.size();
4      vb visited(n, false);
5
6      function<bool(int, int)> dfs = [&](int u, int parent)
7      {
8          visited[u] = true;
9          for (auto v : graph[u])
10             {
11                 if (!visited[v])
12                     return dfs(v,u);
13                 else if (v != parent)
14                     return true;
15             }
16         return false;
17     };
18
19     forn(i,0,n)
20     {
21         if (!visited[i] && dfs(i, -1))
22             return true;
23     }
24     return false;
25 }
```

19.5.3 Bipartito

Un grafo es bipartito si sus nodos pueden colorearse con dos colores de forma que ningún par de nodos adyacentes tenga el mismo color.

```
1  bool bipartite(vvi &graph)
2  {
3      int n = graph.size();
4      vi color(n, -1);
5
6      forn(i,0,n)
7      {
8          if (color[i] == -1)
9          {
10             queue<int> q;
11             q.push(i);
12             color[i] = 0;
13
14             while (!q.empty())
15             {
16                 int u = q.front(); q.pop();
17                 for (auto v : graph[u]);
18                 {
19                     if (color[v] == -1)
20                     {
21                         color[v] = 1 - color[u];
22                         q.push(v);
23                     }
24                     else if (color[v] == color[u])
25                         return false;
26                 }
27             }
28         }
29     }
30     return true;
31 }
```

19.6 Camino más corto

19.6.1 Dijkstra

Dijkstra calcula las distancias más cortas desde un nodo inicial hasta todos los demás en un grafo con pesos no negativos.

En cada paso elige el nodo con menor distancia actual, lo marca como procesado y actualiza las distancias de sus vecinos si encuentra un camino más corto. El proceso termina cuando todos los nodos han sido procesados.

```
1  const int INF = 1e9;
2  vi dijkstra(int start, vvpi &graph)
3  {
4      int n = graph.size();
5      vi vis(n, false), dist(n, INF);
6      priority_queue<pi> q;
7
8      dist[start] = 0;
9      q.push({0, start});
10
11     while (!q.empty()) {
12         int u = q.top().second; q.pop();
13
14         if (vis[u]) continue;
15         vis[u] = true;
16
17         for (auto [w, u] : adj[u]) {
18
19             if (dist[u] + w < dist[v])
20             {
21                 dist[v] = dist[u] + w;
22                 q.push({-dist[v], v});
23             }
24         }
25     }
26     return dist;
27 }
28 }
```

19.6.2 Bellman-Ford

El algoritmo de Bellman-Ford calcula las distancias más cortas desde un nodo inicial incluso si existen aristas con pesos negativos.

Repite $n - 1$ veces la relajación de todas las aristas, actualizando las distancias si encuentra un camino más corto. Si en una ronda adicional alguna distancia aún se reduce, significa que existe un ciclo negativo.

```
1 using viii = vector<tuple<int,int,int>>;
2 vi bellmanFord(int start, int n, viii&adj)
3 {
4     vi dist(n, INF);
5
6     dist[start] = 0;
7
8     forn(i,0,n-1)
9     {
10         for (auto e : adj)
11         {
12             int u, v, w;
13             tie(u, v, w) = e;
14             dist[v] = min(dist[v], dist[u] + w);
15         }
16     }
17
18     return dist;
19 }
```

19.6.3 Floyd-Warshall

Floyd-Warshall encuentra las distancias mínimas entre todos los pares de nodos en un solo recorrido. Usa una matriz de distancias y en cada iteración permite que un nodo intermedio k mejore las rutas existentes. Su simplicidad lo hace ideal para grafos pequeños donde $O(n^3)$ es aceptable.

```
1 using viii = vector<tuple<int,int,int>>;
2
3 vvi floydWarshall(vvi &adj, int n)
4 {
5     vvi d(n, vi(n, INF));
6
7     forn(i,0,n)
8     {
9         forn(j,0,n)
10        {
11            if (i == j) d[i][j] = 0;
12            else if (adj[i][j]) d[i][j] = d[i][j];
13        }
14    }
15
16    forn(k,0,n)
17        forn(i,0,n)
18            forn(j,0,n)
19                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
20
21    return d;
22 }
```

19.6.4 Topological Sort

Topological sort ordena los nodos de un grafo dirigido de forma que si existe un camino de a a b , entonces a aparece antes que b en la lista; existe solo si el grafo es acíclico.

Se realiza con una búsqueda en profundidad (DFS) donde cada nodo tiene tres estados:

0 = no visitado, 1 = en proceso, 2 = procesado.

Cuando un nodo pasa a estado 2 se agrega a la lista; si durante la DFS se encuentra un nodo en estado 1, se detecta un ciclo.

Al final, la lista se invierte para obtener el orden topológico. $O(n + m)$

```
1 vi topologicalSort(vvi &graph, int n)
2 {
3     vi state(n, 0), order;
4     bool has_cycle = false;
5
6     function<void(int)> dfs = [&](int u)
7     {
8         state[u] = 1;
9         for (int v : graph[u])
10        {
11            if (state[v] == 0) dfs(v);
12            else if (state[v] == 1) has_cycle = true;
13        }
14        state[u] = 2;
15        order.push_back(u);
16    };
17
18    forn(i, 0, n)
19        if (state[i] == 0) dfs(i);
20
21    if (has_cycle) return {};
22
23    reverse(all(order));
24    return order;
25 }
```

19.7 Dynamic Programming for Graphs

La programación dinámica en grafos permite responder preguntas sobre caminos en grafos dirigidos acíclicos, como el camino más corto o más largo entre dos nodos, el número total de caminos posibles o qué nodos aparecen en todos ellos.

Por ejemplo, para contar los caminos desde un nodo inicial a otro nodo, se define una función $paths(x)$ que representa el número de caminos desde el nodo inicial hasta x .

Como caso base, $paths(a) = 1$, y para los demás nodos se cumple la relación:

$$paths(x) = paths(s_1) + paths(s_2) + \dots + paths(s_k)$$

donde $s_1, s_2, \dots, s_k$ son los nodos con aristas hacia x .

Dado que el grafo es acíclico, estos valores pueden calcularse en el orden de un ordenamiento topológico.

Esta técnica también se extiende para representar cualquier problema de programación dinámica como un grafo dirigido acíclico donde cada nodo representa un estado y las aristas indican dependencias entre estados.

$$O(n + m)$$

```
1  vi topo = topologicalSort(graph);
2
3  vi paths(n+1, 0);
4  paths[a] = 1;
5
6  for (int u : topo)
7      for (int v : graph[u])
8          paths[v] += paths[u];
9
10 cout << paths[b];
```

19.8 Sucesor Graphs

Los grafos sucesores (successor graphs) son grafos dirigidos donde cada nodo tiene exactamente un sucesor, formando componentes que consisten en un ciclo y caminos que llevan a él.

Se pueden representar mediante una función $\text{succ}(x)$ que indica el sucesor de cada nodo. Para calcular eficientemente el sucesor después de k pasos, se precálculan los valores $\text{succ}(x, k)$ donde k es potencia de dos usando la recurrencia:

$$\text{succ}(x, k) = \begin{cases} \text{succ}(x), & k = 1 \\ \text{succ}(\text{succ}(x, k/2), k/2), & k > 1 \end{cases}$$

Esto permite calcular cualquier $\text{succ}(x, k)$ descomponiendo k en sumas de potencias de dos, logrando un tiempo de $O(\log k)$ por consulta después de precálculos en $O(n \log u)$.

```
1  int LOG = 20;
2  vvi succ(n, vi(LOG)); // next[i] is the immediate sucesor for i
3
4  forn(i,0,n) succ[i][0] = next[i];
5
6  forn(j,1,LOG)
7      forn(i,0,n)
8          succ[i][j] = succ[succ[i][j-1]][j-1];
9
10 int getKsucesor(int start, int k)
11 {
12     forn(j,0,LOG)
13         if (k & (1 << j))
14             x = succ[x][j];
15     return x;
16 }
```


19.8.1 Deteccion de Ciclos

En un grafo sucesor que consiste en un camino que termina en un ciclo, podemos preguntar: dado un nodo inicial x , ¿cuál es el primer nodo del ciclo y cuántos nodos tiene el ciclo?

Una forma eficiente de detectar el ciclo es el algoritmo de Floyd: usamos dos punteros, a y b , que comienzan en el nodo inicial x ; a avanza un paso por turno y b avanza dos pasos. Cuando se encuentran, sabemos que hay un ciclo.

Para encontrar el primer nodo del ciclo, movemos a de nuevo al nodo inicial x y avanzamos ambos punteros paso a paso hasta que se encuentren.

Finalmente, para obtener la longitud del ciclo, recorremos el ciclo desde ese nodo hasta volver a él, contando los nodos. $O(n)$ tiempo y $O(1)$ memoria.

```
1  int findCycle(int x, vi &succ, int &len)
2  {
3      int a = succ[x];
4      int b = succ[succ[x]];
5
6      while (a != b)
7      {
8          a = succ[a];
9          b = succ[succ[b]];
10     }
11
12     a = x;
13     while (a != b)
14     {
15         a = succ[a];
16         b = succ[b];
17     }
18
19     int first = a;
20     b = succ[first];
21     len = 1;
22
23     while (b != first)
24     {
25         b = succ[b];
26         len++;
27     }
28
29     return first;
30 }
```

19.8.2 Puentes

Un puente en un grafo es una arista cuya eliminación dividiría el grafo en dos o más componentes conexas adicionales.

```
1  vector<pii> find_bridges(vvi &g)
2  {
3      int n = g.size(), t = 0;
4      vi tin(n, -1), low(n, -1);
5      vector<pii> bridges;
6
7      function<void(int,int)> dfs = [&](int u, int p)
8      {
9          tin[u] = low[u] = t++;
10         for (int v : g[u])
11         {
12             if (v == p) continue;
13             if (tin[v] != -1)
14                 low[u] = min(low[u], tin[v]);
15             else
16             {
17                 dfs(v, u);
18                 low[u] = min(low[u], low[v]);
19                 if (low[v] > tin[u])
20                     bridges.push_back({u,v});
21             }
22         }
23     };
24
25     forn(i,n) if (tin[i] == -1) dfs(i, -1);
26     return bridges;
27 }
```

20 Minimum Spanning Tree

Un árbol generador (spanning tree) contiene todos los nodos de un grafo y algunas de sus aristas de modo que exista un camino entre cualquier par de nodos.

Un árbol generador mínimo (minimum spanning tree, MST) es aquel cuyo peso total de aristas es el menor posible, mientras que un árbol generador máximo tiene el peso total mayor.

20.0.1 Union-Find / Disjoint Sets

La estructura Union-Find mantiene conjuntos disjuntos y permite unir conjuntos y encontrar el representante de un elemento en $O(\log n)$ tiempo.

Cada conjunto tiene un representante y los elementos apuntan a él siguiendo un camino. Al unir conjuntos, siempre se conecta el representante del conjunto más pequeño al del más grande para mantener los caminos cortos.

Con compresión de caminos (path compression), cada elemento apunta directamente a su representante tras una búsqueda, logrando operaciones en tiempo amortizado $O(\alpha(n))$, donde α es la inversa de Ackermann.

```
1  class Dsu
2  {
3  public:
4      vi p, rank; // Parent, Rank
5
6      Dsu(int n)
7      {
8          p.resize(n);
9          rank.resize(n, 1);
10         forn(i, 0, n) p[i] = i;
11     }
12
13     int find(int x)
14     {
15         if (x == p[x]) return x;
16         return find(p[x]);
17     }
18
19     // Path Compression
20     int findPc(int x)
21     {
22         if (x == p[x]) return x;
23         return p[x] = findPc(p[x]);
24     }
25
26     bool same(int a, int b)
27     {
28         return find(a) == find(b);
29     }
30
31     void unite(int a, int b)
32     {
33         a = find(a), b = find(b);
34         p[b] = a;
35     }
36     void uniteRank(int a, int b)
37     {
38         a = find(a), b = find(b);
39         if (a==b) return;
40
41         if (rank[a] < rank[b]) swap(a,b);
42         p[b] = a;
43         if (rank[a] == rank[b]) rank[a]++;
44     }
45 };
```

20.0.2 Kruskal

Kruskal construye un MST usando un enfoque greedy: primero ordena las aristas por peso y luego agrega cada arista que une dos componentes distintas. Esto garantiza que no se formen ciclos y que siempre se agreguen las aristas de menor peso primero, asegurando la optimalidad del MST.

Para implementar eficientemente las funciones same (verificar si dos nodos a y b están en la misma componente) y

unite (unir componentes), se usa la estructura union-find o disjoint set, que permite ejecutar ambas operaciones en $O(\log n)$.

El tiempo total de Kruskal es $O(m \log m)$ para ordenar las aristas y $O(m \log n)$ para procesarlas usando union-find.

```
1 using iii = tuple<int, int, int>;
2 using viii = vector<iii>;
3 viii kruskal(int n, viii adj, int &total)
4 {
5     auto cmp = [](iii a, iii b)
6     { return get<2>(a) < get<2>(b) };
7
8     sort(all(adj), cmp);
9
10    Dsu dsu(n);
11    total = 0;
12    viii mst;
13
14    for (auto [u, v, w] : adj)
15    {
16        if (!dsu.same(u, v))
17        {
18            dsu.unite(u, v);
19            total += w;
20            mst.push_back({u, v, w});
21        }
22    }
23    return mst;
24 }
```

20.0.3 Prim

Prim's algorithm construye un árbol de expansión mínima agregando nodos uno por uno.

Se inicia con un nodo arbitrario y, en cada paso, se elige la arista de menor peso que conecta un nodo ya incluido con uno fuera del árbol.

El proceso continúa hasta que todos los nodos estén agregados, asegurando que el árbol resultante sea mínimo.

La implementación eficiente usa una cola de prioridad que contiene todas las aristas que pueden agregar un nodo nuevo, ordenadas por peso.

En cada iteración se extrae la arista más ligera, se agrega al árbol si conecta nodos distintos y se actualizan las posibles aristas siguientes.

El algoritmo tiene complejidad $O(n + m \log m)$ y produce el mismo resultado que Kruskal, aunque puede ser más práctico para grafos densos o cuando se conoce un nodo inicial.

```
1
2 using iii = tuple<int, int, int>;
3 using viii = vector<iii>;
4 viii prim(int n, viii &adj, int &total, int start = 0)
5 {
6     vi vis(n, false);
7     priority_queue<iii, viii, greater<>> pq;
8
9     // {weight, current, parent}
10    pq.push({0, start, start});
11
12    viii mst;
13    total = 0;
14
15    while (!pq.empty())
16    {
17        auto [w, u, p] = pq.top(); pq.pop();
18        if (vis[u]) continue;
19        vis[u] = true;
20
21        if (u != start)
22            mst.push_back({p, u, w});
23        total += w;
24
25        for (auto [v, wt] : adj[u])
26        {
27            if (!vis[v])
28                pq.push({wt, v, u});
29        }
30    }
31    return mst;
32 }
```

20.0.4 Strong Connectivity

20.0.5 Kosaraju

El algoritmo de **Kosaraju** encuentra las **componentes fuertemente conexas (SCC)** en un grafo dirigido, donde cada componente es un conjunto de nodos que se alcanzan mutuamente. El proceso tiene dos fases:

- 1 Realiza un DFS normal y guarda los nodos según su orden de finalización.
- 2 Invierte las aristas y ejecuta DFS en ese orden inverso para formar las SCC.

Cada nodo se visita exactamente dos veces, por lo que su complejidad es $O(n + m)$.

```
1 #define pb push_back
2 vvi kosaraju(vvi &g, int n)
3 {
4     vvi rg(n), scc;
5     vb vis(n,0);
6     vi order;
7
8     forn(u,0,n)
9         for (int v : g[u])
10             rg[v].pb(u);
11
12     function<void(int)> dfs1 = [&](int u)
13     {
14         vis[u] = 1;
15         for (int v : g[u])
16             if (!vis[v]) dfs1(v);
17         order.pb(u);
18     };
19
20     function<void(int,vi&)> dfs2 = [&](int u, vi &comp)
21     {
22         vis[u] = 1;
23         comp.pb(u);
24         for (int v : rg[u])
25             if (!vis[v]) dfs2(v, comp);
26     };
27
28     forn(i,0,n)
29         if (!vis[i]) dfs1(i);
30
31     fill(all(vis),0);
32     reverse(all(order));
33
34
35     for (int u : order)
36         if (!vis[u])
37         {
38             vi comp;
39             dfs2(u, comp);
40             scc.pb(comp);
41         }
42
43     return scc;
44 }
45 }
```

20.0.6 2 SAT

20.0.7 2-SAT

El **problema 2-SAT** consiste en determinar si es posible asignar valores de verdad a un conjunto de variables booleanas $x_1, x_2, \dots, x_n$ para que una fórmula de la forma $(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \dots \wedge (a_m \vee b_m)$ sea verdadera, donde cada a_i y b_i son una variable o su negación.

Cada cláusula $(a \vee b)$ genera dos **implicaciones**: $\neg a \rightarrow b$ y $\neg b \rightarrow a$. Con esto, construimos un **grafo de implicaciones** y verificamos si existe una variable x_i y su negación $\neg x_i$ en la misma **componente fuertemente conexa (SCC)**. Si eso ocurre, la fórmula es **insatisfacible**. Si no ocurre, la fórmula **tiene solución**, que se obtiene procesando las componentes en orden topológico inverso. Su complejidad es $O(n + m)$.

20.0.8 2-SAT

El **problema 2-SAT** busca determinar si se pueden asignar valores de verdad a las variables booleanas $x_1, x_2, \dots, x_n$ para que $(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \dots \wedge (a_m \vee b_m)$ sea verdadera, donde cada a_i y b_i es una variable o su negación.

Por ejemplo, la fórmula

$$(x_2 \vee \neg x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_4)$$

es verdadera cuando las variables toman los valores:

$$\begin{cases} x_1 = \text{false} \\ x_2 = \text{false} \\ x_3 = \text{true} \\ x_4 = \text{true} \end{cases}$$

Cada cláusula $(a \vee b)$ genera dos **implicaciones**: $\neg a \rightarrow b$ y $\neg b \rightarrow a$. Con esto se construye un **grafo de implicaciones**, donde el problema es **insatisfacible** si existe una variable x_i y su negación $\neg x_i$ en la misma **componente fuertemente conexa (SCC)**.

Si no ocurre, la fórmula tiene solución, y los valores se obtienen procesando las componentes en orden topológico inverso. Su complejidad es $O(n + m)$.

```

1 struct TwoSAT
2 {
3     int n;
4     vvi g, rg;
5     vb val, vis;
6     vi order, comp;
7     TwoSAT(int n) : n(n), g(2*n), rg(2*n),
8                     val(n, 0), vis(2*n, 0),
9                     comp(2*n, -1) {}
10
11     int neg(int x) { return x ^ 1; }
12
13     void addClause(int a, bool na, int b, bool nb)
14     {
15         a = 2*a ^ na;
16         b = 2*b ^ nb;
17
18         g[neg(a)].pb(b);
19         g[neg(b)].pb(a);
20
21         rg[b].pb(neg(a));
22         rg[a].pb(neg(b));
23     }
24
25     void dfs1(int u)
26     {
27         vis[u] = 1;
28         for (int v : g[u])
29             if (!vis[v]) dfs1(v);
30         order.pb(u);
31     }
32
33     void dfs2(int u, int c)
34     {
35         comp[u] = c;
36         for (int v : rg[u])
37             if (comp[v] == -1) dfs2(v, c);
38     }
39
40     bool solve()
41     {
42         forn(i, 0, 2*n)
43             if (!vis[i]) dfs1(i);
44
45         reverse(all(order));
46         int c = 0;
47
48         for (int u : order)
49             if (comp[u] == -1) dfs2(u, c++);
50
51         forn(i, 0, n)
52         {
53             if (comp[2*i] == comp[2*i^1])
54                 return 0;
55             val[i] = comp[2*i] > comp[2*i^1];
56         }
57         return 1;
58     }
59 };

```

20.0.9 Eulerian Paths and Circuits

Un **camino Euleriano** es aquel que recorre **cada arista exactamente una vez**. Si el camino empieza y termina en el mismo nodo, se llama **circuito Euleriano**.

En **grafos no dirigidos**, existe un camino Euleriano si todas las aristas están en la misma componente conexa y:

- Todos los nodos tienen grado par, o
- Exactamente dos nodos tienen grado impar (estos serán los extremos del camino).

En **grafos dirigidos**, se cumple si todas las aristas pertenecen a la misma componente y:

- Cada nodo tiene el mismo número de entradas y salidas (circuito Euleriano), o
- Hay un nodo con una salida más que entradas (inicio) y otro con una entrada más que salidas (final).

Para **construir** un circuito o camino Euleriano se usa el **algoritmo de Hierholzer**, que:

1. Empieza desde un nodo con aristas disponibles.
2. Recorre aristas sin repetir hasta regresar al inicio (subcircuito).
3. Inserta subcircuitos dentro del circuito principal hasta usar todas las aristas.

Su complejidad es $O(n + m)$, ya que cada arista se recorre una vez.

```

1  vi eulerianPath(vvi &graph, int n)
2  {
3      vi indeg(n, 0), outdeg(n, 0), path;
4      vpi st;
5
6      int start = 0, end = 0, cnt = 0;
7
8      forn(u, 0, n)
9          for (int v : adj[u])
10             {
11                 outdeg[u]++;
12                 indeg[v]++;
13             }
14
15     forn(i, 0, n)
16     {
17         if (outdeg[i] - indeg[i] == 1)
18             start = i, cnt++;
19         else if (indeg[i] - outdeg[i] == 1)
20             end = i, cnt++;
21     }
22
23     if (cnt != 0 && cnt != 2)
24         return {};
25
26     st.push_back({start, 0});
27
28     while (!st.empty())
29     {
30         auto &[u, i] = st.back();
31
32         if (i < graph[u].size())
33         {
34             int v = graph[u][i++];
35             st.push_back({v, 0});
36         }
37         else
38         {
39             path.pb(u);
40             st.pop_back();
41         }
42     }
43
44     reverse(all(path));
45     return path;
46 }

```

20.0.10 Hamiltonian Path

El **camino Hamiltoniano** recorre **cada vértice exactamente una vez**, y si empieza y termina en el mismo vértice, se denomina **circuito Hamiltoniano**.

Encontrar uno es un problema **NP-hard**, ya que no existe un algoritmo eficiente general. Sin embargo, usando **programación dinámica con bitmask**, se puede verificar su existencia en tiempo $O(2^n n^2)$.

$$dp[mask][u] = \bigvee_{v \in mask, adj[v][u]} dp[mask \setminus \{u\}][v]$$

```

1  vi hamiltonian_path(vvi &graph)
2  {
3      int n = graph.size();
4      vvi dp(1<<n, vi(n, -1)), par(1<<n, vi(n, -1));

```

```

5
6 function<int(int,int)> dfs = [&](int m, int u)
7 {
8     if(m==(1<<n)-1) return 1;
9     int &r=dp[m][u];
10    if(r!=-1) return r;
11    r=0;
12    forn(v,0,n) if(graph[u][v] && !(m&(1<<v)))
13        if(dfs(m|(1<<v), v))
14            { par[m][u]=v; r=1; break; }
15    return r;
16 };
17
18 forn(s,0,n) if(dfs(1<<s,s))
19 {
20     vi p={s};
21     int m=1<<s,u=s;
22     while(par[m][u]!=-1)
23         { u=par[m][u]; m|=1<<u; p.push_back(u); }
24     return p;
25 }
26 return {};
27 }

```

20.0.11 De Bruijn Sequences

Una **secuencia de De Bruijn** es una cadena que contiene **todas las posibles subcadenas de longitud n** exactamente una vez, usando un alfabeto de k símbolos.

La longitud total es $k^n + n - 1$.

Por ejemplo, con $n = 3$ y $k = 2$, una secuencia válida es `0001011100`,

que incluye todas las combinaciones binarias de 3 bits: 000, 001, 010, 011, 100, 101, 110 y 111.

Cada secuencia De Bruijn corresponde a un **camino Euleriano** en un grafo donde cada nodo representa una cadena de $n - 1$ símbolos y cada arista agrega uno nuevo.

Al recorrer el camino Euleriano y concatenar los símbolos, se obtiene la secuencia completa de longitud $k^n + n - 1$.

```
1  string debruijn(int k, int n)
2  {
3      vi a(k*n,0);
4      string s;
5      function<void(int,int)> db=[&](int t,int p)
6      {
7          if(t>n)
8          {
9              if(n%p==0) for(j,1,p+1) s.push_back('0'+a[j]);
10             }
11             else
12             {
13                 a[t]=a[t-p];
14                 db(t+1,p);
15                 for(j,a[t-p]+1,k-1) { a[t]=j; db(t+1,t); }
16             }
17         };
18         db(1,1);
19         s.resize(s.size() + n - 1, '0');
20         return s;
21     }
```

20.0.12 Knight's Tour

Un recorrido del caballo es una secuencia de movimientos en un tablero nn donde el caballo visita cada casilla exactamente una vez. Si termina en el punto de inicio, se llama recorrido cerrado; de lo contrario, es abierto. El problema equivale a encontrar un camino Hamiltoniano en el grafo formado por las casillas como nodos y los movimientos válidos del caballo como aristas.

El recorrido puede generarse por backtracking, pero se acelera con la regla de Warnsdorf, que elige siempre el movimiento hacia la casilla con el menor número de movimientos siguientes posibles.

```
1  int dx[8]={1,2,2,1,-1,-2,-2,-1},dy[8]={2,1,-1,-2,-2,-1,1,2};
2
3  bool knightsTour(int n, vvi &b, int x, int y, int mv)
4  {
5      if(mv==n*n) return true;
6      vector<pair<int,pi>> nxt;
7      forn(d,0,8)
8      {
9          int nx=x+dx[d],ny=y+dy[d];
10         if(nx<0||ny<0||nx>=n||ny>=n||b[nx][ny]!=-1) continue;
11         int c=0;
12         forn(k,0,8)
13         {
14             int tx=nx+dx[k],ty=ny+dy[k];
15             if(tx>=0&&ty>=0&&tx<n&&ty<n&&b[tx][ty]==-1) c++;
16         }
17         nxt.push_back({c,{nx,ny}});
18     }
19     sort(nxt.begin(),nxt.end());
20     for(auto [c,p]:nxt)
21     {
22         auto [nx,ny]=p;
23         b[nx][ny]=mv;
24         if(knightsTour(n,b,nx,ny,mv+1)) return true;
25         b[nx][ny]=-1;
26     }
27     return false;
28 }
```

21 Flujos

21.1 Red de Flujos

Una red de flujo es un grafo dirigido con:

- V vértices y E aristas
- Cada arista tiene una capacidad $c(u,v) \geq 0$
- Hay una fuente s y un sumidero t

Un flujo $f(u,v)$ cumple:

- 1) $0 \leq f(u,v) \leq c(u,v)$
- 2) Conservación del flujo en todo vértice excepto s y t : suma de flujos que entran = suma de flujos que salen

Objetivo: Enviar todo el flujo posible desde s hasta t .

21.2 Grafo Residual

El grafo residual contiene:

- Aristas con capacidad disponible: $\text{residual} = c(u,v) - f(u,v)$
- Aristas reversas con $\text{residual} = f(u,v)$

Si existe flujo hacia adelante, puede enviarse más; si existe flujo hacia atrás, puede revertirse parte del flujo.

Este grafo cambia dinámicamente durante el algoritmo, para la lectura se tienen que realizar algunas modificaciones.

```
1 vvi graph(n), cap(n, vi(n));
2 int u,v,w;
3 forn(i,0,m)
4 {
5     cin>>u>>v>>w;
6     graph[u].push_back(v);
7     graph[v].pus_back(u);
8     cap[u][v] += w;
9 }
```

21.3 Ford-Fulkerson

Idea: Mientras exista un camino aumentante de $s \rightarrow t$ en el grafo residual:

- Se calcula el mínimo residual C en ese camino
- Se aumenta el flujo en C a lo largo del camino
- Se actualizan capacidades residuales

Termina cuando ya no hay camino aumentante. Complejidad: $O(E * F)$ donde F = flujo máximo (Si las capacidades son enteras, siempre termina)

```
1  int fordFulkerson(vvi &graph, vvi &cap, int s, int t)
2  {
3      int n = graph.size(), flow = 0;
4
5      function<int(int,int,int,vi)> dfs = [&](int u, int t, int f, vi &vis) {
6          if (u == t) return f;
7          vis[u] = 1;
8          for (int v : graph[u])
9              {
10                 if (!vis[v] && cap[u][v] > 0)
11                     {
12                         int pushed = dfs(v, t, min(f, cap[u][v]), vis);
13                         if (pushed > 0) {
14                             cap[u][v] -= pushed;
15                             cap[v][u] += pushed;
16                             return pushed;
17                         }
18                     }
19             }
20         return 0;
21     };
22
23     while (true)
24     {
25         vi vis(n);
26         int pushed = dfs(s, t, 1e9, vis);
27         if (!pushed) break;
28         flow += pushed;
29     }
30     return flow;
31 }
```

21.4 Edmonds-Karps

Mejora de Ford–Fulkerson: usar BFS para encontrar caminos aumentantes → Elige siempre el camino más corto en número de aristas

Complejidad garantizada: $O(V * E^2)$ incluso con capacidades irracionales

Es el algoritmo estándar para Max-Flow en competencias.

```
1  int edmondsKarp(vvi &graph, vvi &cap, int s, int t)
2  {
3      int n = graph.size(), flow = 0;
4      while (true)
5      {
6          vi parent(n, -1);
7          parent[s] = -2;
8          queue<pi> q;
9          q.push({s, 1e9});
10         int pushed = 0;
11
12         while (!q.empty())
13         {
14             auto [u, f] = q.front(); q.pop();
15             for (int v : graph[u])
16             {
17                 if (parent[v] == -1 && cap[u][v] > 0)
18                 {
19                     parent[v] = u;
20                     int new_flow = min(f, cap[u][v]);
21                     if (v == t) { pushed = new_flow; break; }
22                     q.push({v, new_flow});
23                 }
24             }
25             if (pushed) break;
26         }
27         if (!pushed) break;
28
29         int v = t;
30         while (v != s)
31         {
32             int u = parent[v];
33             cap[u][v] -= pushed;
34             cap[v][u] += pushed;
35             v = u;
36         }
37         flow += pushed;
38     }
39     return flow;
40 }
```


21.5 Teorema del Corte Minimo

Teorema: El valor del flujo máximo es igual al valor del mínimo corte

Un s-t cut es una partición de los vértices:

- S contiene a s
- T contiene a t Se suman capacidades de aristas de $S \rightarrow T$

No se puede mandar más flujo del valor del corte mínimo y Ford–Fulkerson demuestra que sí se puede llegar a ese valor.

```
1 pair<vpi,int> minCut(vvi &graph, vvi &cap, int s, int t)
2 {
3     int n = graph.size(), sum = 0;
4     vvi orig = cap;
5     vpi vis(n, 0);
6     vpi cut;
7     queue<int> q;
8
9
10    edmondsKarp(graph, cap, s, t);
11
12    q.push(s);
13    vis[s] = 1;
14    while (!q.empty())
15    {
16        int u = q.front(); q.pop();
17        for (int v : graph[u])
18        {
19            if (!vis[v] && cap[u][v] > 0)
20            {
21                vis[v] = 1;
22                q.push(v);
23            }
24        }
25    }
26
27    forn(u,0,n) if (vis[u])
28    {
29        for (int v : graph[u])
30        {
31            if (!vis[v] && orig[u][v] > 0)
32            {
33                cut.eb(u,v);
34                sum += orig[u][v];
35            }
36        }
37    }
38    return {cut, sum};
39 }
40 pair<vpi,int> minCut(vvi &graph, vvi &cap, int s, int t)
41 {
42     int n = graph.size(), sum = 0;
43     vvi orig = cap;
44     vpi cut;
45     vpi vis(n, 0);
46     queue<int> q;
47
48    edmondsKarp(graph, cap, s, t);
49
50    q.push(s);
51    vis[s] = 1;
52    while (!q.empty())
53    {
```

```

54     int u = q.front(); q.pop();
55     for (int v : graph[u])
56     {
57         if (!vis[v] && cap[u][v] > 0)
58         {
59             vis[v] = 1;
60             q.push(v);
61         }
62     }
63 }
64
65 forn(u,0,n) if (vis[u])
66 {
67     for (int v : graph[u])
68     {
69         if (!vis[v] && orig[u][v] > 0)
70         {
71             cut.eb(u,v);
72             sum += orig[u][v];
73         }
74     }
75 }
76 return {cut, sum};
77 }

```

21.5.1 Disjoint Paths

Los caminos disjuntos en aristas se encuentran asignando capacidad 1 a cada arista y calculando el flujo máximo. Cada unidad de flujo representa un camino independiente entre la fuente y el sumidero.

Para los caminos disjuntos en nodos, se divide cada vértice en dos (entrada y salida) conectados por una arista con capacidad 1, asegurando que cada nodo solo pueda usarse en un camino. El máximo flujo en esta red equivale al número máximo de caminos nodo-disjuntos.

```

1  int nodeDisjointPaths(int n, vpi &edges, int s, int t)
2  {
3      int m = 2*n;
4      vvi graph(m), cap(N, vi(m, 0));
5      forn(i,0,n)
6      {
7          graph[i].pb(i+n);
8          cap[i][i+n] = 1;
9      }
10     for (auto [u,v] : edges)
11     {
12         graph[u+n].pb(v);
13         graph[v].pb(u+n);
14         cap[u+n][v] = 1;
15     }
16     return edmondsKarp(graph, cap, s+n, t);
17 }

```

21.6 Bipartite Matching

El emparejamiento máximo bipartito se reduce fácilmente a un problema de flujo máximo: se conectan todos los nodos izquierdos con la fuente y todos los derechos con el sumidero, asignando capacidad 1 a cada arista. El valor del flujo máximo equivale al tamaño del emparejamiento máximo.

El teorema de Hall establece cuándo existe un emparejamiento perfecto: para todo subconjunto de nodos izquierdos X , su conjunto de vecinos $f(X)$ debe cumplir $|X| \leq |f(X)|$. Si alguna elección viola esta condición, no puede formarse un emparejamiento perfecto.

```

1  int bipartiteMatching(int nLeft, int nRight, vector<pi> &edges)
2  {
3      int n = nLeft + nRight + 2;
4      int s = nLeft + nRight;
5      int t = s + 1;
6
7      vvi graph(n);
8      vvi cap(n, vi(n, 0));
9
10     // conexiones desde fuente a nodos izquierdos
11     forn(i,0,nLeft)
12     {
13         graph[s].pb(i);
14         graph[i].pb(s);
15         cap[s][i] = 1;
16     }
17
18     // conexiones entre conjuntos izquierdo y derecho
19     for (auto [u,v] : edges)
20     {
21         int left = u;
22         int right = nLeft + v;
23         graph[left].pb(right);
24         graph[right].pb(left);
25         cap[left][right] = 1;
26     }
27
28     // conexiones desde nodos derechos al sumidero
29     forn(i,0,nRight)
30     {
31         int r = nLeft + i;
32         graph[r].pb(t);
33         graph[t].pb(r);
34         cap[r][t] = 1;
35     }
36
37     return edmondsKarp(graph, cap, s, t);
38 }

```

21.7 Puentes

Un puente en un grafo es una arista cuya eliminación dividiría el grafo en dos o más componentes conexas adicionales.

```
1 void findBridges(vvi &graph)
2 {
3     int n = graph.size();
4     int timer = 0;
5     vb vis(n, false);
6     vi tin(n, -1), low(n, -1);
7
8     auto IS_BRIDGE = [&](int u, int v)
9     {
10         cout << u << " " << v << "\n";
11     };
12
13     function<void(int,int)> dfs = [&](int u, int p)
14     {
15         vis[u] = true;
16         tin[u] = low[u] = timer++;
17         for (int v : graph[u])
18         {
19             if (v == p) continue;
20             if (vis[v]) low[u] = min(low[u], tin[v]);
21             else
22             {
23                 dfs(v, u);
24                 low[u] = min(low[u], low[v]);
25                 if (low[v] > tin[u]) IS_BRIDGE(u, v);
26             }
27         }
28     };
29
30     forn(i,0,n)
31         if (!vis[i])
32             dfs(i, -1);
33 }
```

21.8 Puntos de articulacion

```
1 void findArticulationPoints(vvi &graph)
2 {
3     int n = graph.size();
4     int timer = 0;
5     vb vis(n, false);
6     vi tin(n, -1), low(n, -1);
7
8     auto IS_CUTPOINT = [&](int u)
9     {
10         cout << u << "\n";
11     };
12
13     function<void(int,int)> dfs = [&](int u, int p)
14     {
15         vis[u] = true;
16         tin[u] = low[u] = timer++;
17         int ch = 0;
18
19         for (int v : graph[u])
20         {
21             if (v == p) continue;
22             if (vis[v]) low[u] = min(low[u], tin[v]);
23             else
```

```

24         {
25             dfs(v, u);
26             low[u] = min(low[u], low[v]);
27             if (low[v] >= tin[u] && p != -1) IS_CUTPOINT(u);
28             ++ch;
29         }
30     }
31
32     if (p == -1 && ch > 1) IS_CUTPOINT(u);
33 };
34
35 forn(i, 0, n)
36     if (!vis[i])
37         dfs(i, -1);
38 }

```

21.9 Patrones y Diseño de Algoritmos

21.9.1 Hamming Distance

La distancia de Hamming entre dos cadenas a y b de igual longitud es el número de posiciones donde difieren. Por ejemplo, $\text{hamming}(01101, 11001) = 2$.

Si representamos las cadenas como enteros (cuando $k \leq 32$), podemos calcular la distancia usando operaciones a nivel de bit. El XOR ($\oplus$) identifica los bits diferentes y contar los 1s.

Esto reduce la complejidad de $O(n^2k)$ a $O(n^2)$ al comparar n cadenas de longitud k .

```

1  int hamming(int a, int b)
2  {
3      bitset<32> x(a), y(b);
4      return (x ^ y).count();
5  }
6
7  int hamming(string a, string b)
8  {
9      int d = 0;
10     forn(i, 0, a.size())
11         if (a[i] != b[i]) d++;
12     return d;
13 }

```

21.9.2 Reachability in Graphs

Dado un DAG con n nodos, queremos calcular $reach(x)$: número de nodos alcanzables desde x .

Se puede usar programación dinámica clásica $O(n^2)$ o bit-paralela usando `bitset` para cada nodo, donde `reach[x] |= reach[u]` para cada u adyacente a x . Esto reduce operaciones por nodo usando OR de bits, pero puede usar mucha memoria ($O(n^2)$ bits).

```
1 using vbt = vector<bitset<1000>>;
2 vbt computeReach(vvi &graph, int n)
3 {
4     vbt reach;
5     for (int x = n-1; x >= 0; x--)
6     {
7         reach[x][x] = 1;
8         for (auto u : adj[x])
9             reach[x] |= reach[u];
10    }
11    return reach;
12 }
```

21.9.3 Counting Subgrids

Dado un grid $n \times n$ con casillas negras (1) o blancas (0), queremos contar subgrids cuyos cuatro vértices son negros.

En la implementación bit-paralela, cada fila se representa como un `bitset` y usamos operaciones AND para contar columnas donde ambas filas tienen negro. La complejidad baja de $O(n^3)$ a $O(n^2)$ y es mucho más rápida, pero usa más memoria ($O(n^2)$ bits para los bitsets).

```
1 using vbt = vector<bitset<1000>>;
2 int countSubgrids(vbt &row, int n) {
3     int total = 0;
4     forn (a, 0, n) {
5         for (b, a+1, n) {
6             int c = (row[a] & row[b]).count();
7             total += c * (c-1) / 2;
8         }
9     }
10    return total;
11 }
```

21.9.4 Two Pointers Method

Se usan dos punteros l y r que solo avanzan hacia la derecha para encontrar subarrays con ciertas propiedades (ej: suma = x).

Ejemplo clásico: encontrar subarray con suma exacta x . Ambos punteros se mueven $O(n)$ pasos en total, dando complejidad $O(n)$.

También se aplica al problema 2SUM: encontrar dos valores cuya suma sea x . Primero se ordena el array $O(n \log n)$ y luego se usan dos punteros desde extremos opuestos.

```
1 bool subarraySum(vi v, int x) {
2     int n = v.size();
3     int l = 0, r = 0, sum = 0;
4     while (r < n) {
5         sum += v[r++];
6         while (sum > x) sum -= v[l++];
7         if (sum == x) return true;
8     }
9     return false;
10 }
11
12 bool twoSum(vi v, int x) {
13     sort(all(v));
14     int l = 0, r = v.size()-1;
15     while (l < r) {
16         int s = v[l] + v[r];
17         if (s == x) return true;
18         else if (s < x) l++;
19         else r--;
20     }
21     return false;
22 }
```

21.9.5 Nearest Smaller Element (Amortized Stack)

Para cada elemento de un array, encontramos el primer elemento más pequeño que lo precede usando un stack.

Cada elemento se agrega exactamente una vez y se elimina a lo sumo una vez $\rightarrow O(n)$ tiempo. Es un ejemplo de análisis amortizado: operaciones individuales varían, pero el total es lineal.

```
1 vi nearestSmaller(vi v)
2 {
3     int n = v.size();
4     vi ans(n, -1);
5     stack<int> st;
6     forn(i, 0, n)
7     {
8         while (!st.empty() && v[st.top()] >= v[i])
9             st.pop();
10        if (!st.empty()) ans[i] = v[st.top()];
11        st.push(i);
12    }
13    return ans;
14 }
```

21.9.6 Sliding Window Minimum

Se mantiene una ventana de tamaño fijo moviéndose sobre un array, y queremos el mínimo de cada ventana.

Se usa un deque: los elementos se agregan al final y se eliminan del final si son mayores al nuevo elemento. El primero siempre es el mínimo. Cada elemento se agrega y elimina a lo sumo una vez $\rightarrow O(n)$.

```

1  vi slidingWindowMin(vi v, int k)
2  {
3      int n = v.size();
4      vi ans;
5      deque<int> dq;
6      forn(i, 0, n)
7      {
8          while (!dq.empty() && dq.front() <= i-k)
9              dq.pop_front();
10         while (!dq.empty() && v[dq.back()] >= v[i])
11             dq.pop_back();
12         dq.push_back(i);
13         if (i >= k-1)
14             ans.push_back(v[dq.front()]);
15     }
16     return ans;
17 }

```

21.9.7 Minimizing Sums

Dado un conjunto de n números $a_1, a_2, \dots, a_n$, consideremos el problema de encontrar un valor x que minimice la suma

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|$$

Por ejemplo, si los números son $[1, 2, 9, 2, 6]$, la solución óptima es $x = 2$, lo que produce la suma

$$|1 - 2| + |2 - 2| + |9 - 2| + |2 - 2| + |6 - 2| = 12$$

Cada función $|a_k - x|$ es convexa, así que la suma también lo es. Podríamos usar búsqueda ternaria para encontrar el x óptimo, pero hay una solución más simple: el valor óptimo de x siempre es la mediana de los números, es decir, el elemento central después de ordenar. Por ejemplo, $[1, 2, 9, 2, 6]$ se ordena como $[1, 2, 2, 6, 9]$, y la mediana es 2.

La mediana siempre es óptima, porque si x es menor que la mediana, la suma disminuye al aumentar x , y si x es mayor, la suma disminuye al disminuir x . Si n es par y hay dos medianas, ambas medianas y todos los valores entre ellas son soluciones óptimas.

Ahora consideremos minimizar la función

$$(a_1 - x)^2 + (a_2 - x)^2 + \dots + (a_n - x)^2$$

Por ejemplo, con $[1, 2, 9, 2, 6]$, la mejor elección es $x = 4$, produciendo la suma

$$(1 - 4)^2 + (2 - 4)^2 + (9 - 4)^2 + (2 - 4)^2 + (6 - 4)^2 = 46$$

Esta función también es convexa. Aunque podríamos usar búsqueda ternaria, hay una solución directa: el valor óptimo de x es el promedio de los números. En este ejemplo, el promedio es

$$(1 + 2 + 9 + 2 + 6)/5 = 4$$

Esto se puede demostrar reescribiendo la suma como

$$nx^2 - 2x(a_1 + a_2 + \dots + a_n) + (a_1^2 + a_2^2 + \dots + a_n^2)$$

La última parte no depende de x , y la función restante forma una parábola con mínimo en $x = s/n$, donde $s = a_1 + a_2 + \dots + a_n$. Por lo tanto, el mínimo se alcanza en el promedio de los números $a_1, a_2, \dots, a_n$.


```

1  int minAbsSum(vi v)
2  {
3      sort(all(v));
4      return v[v.size() / 2];
5  }
6
7  double minSquareSum(vi v)
8  {
9      double a = accumulate(all(v), 0.0);
10     return a / a.size();
11 }

```

22 Range Queries

22.1 Segment Tree

Un segment tree es una estructura de datos flexible que permite responder consultas sobre rangos (como suma, mínimo o máximo) y actualizar valores en un arreglo en tiempo logarítmico. Divide el arreglo en segmentos (intervalos) y almacena información agregada de cada uno, lo que permite combinar resultados de subsegmentos para responder consultas sobre cualquier rango. Es ideal cuando hay muchas actualizaciones entre consultas.

```

1  struct stnode
2  { /* TODO: */ };
3
4  stnode identity; // neutral element for combine
5
6  stnode combine(const stnode &a, const stnode &b)
7  {
8      // TODO: merge logic
9      return stnode();
10 }
11
12 struct SegTree
13 {
14     int n;
15     vector<stnode> st;
16
17     SegTree(int n = 0)
18     { init(n); }
19
20     void init(int s)
21     { n = s; st.assign(2 * n, identity); }
22
23     void build(vector<stnode> &a)
24     {
25         forn (i, 0, n)
26             st[n + i] = a[i];
27
28         for (int i = n - 1; i > 0; i--)
29             st[i] = combine(st[i << 1], st[i << 1 | 1]);
30     }
31
32     void update(int p, stnode val)
33     {
34         for (st[p += n] = val; p > 1; p >>= 1)
35             st[p >> 1] = combine(st[p], st[p ^ 1]);
36     }
37
38     stnode query(int l, int r) // [l, r)
39     {
40         stnode resL = identity, resR = identity;
41

```

```

42     for (l += n, r += n; l < r; l >>= 1, r >>= 1)
43     {
44         if (l & 1) resL = combine(resL, st[l++]);
45         if (r & 1) resR = combine(st[--r], resR);
46     }
47
48     return combine(resL, resR);
49 }
50 };

```

22.2 Sparse Table

Una sparse table también sirve para responder consultas sobre rangos, pero es inmutable: solo funciona cuando el arreglo no cambia. Precalcula todas las respuestas posibles para rangos de tamaño potencia de dos, lo que permite responder consultas en tiempo constante combinando solo dos intervalos. Es perfecta para consultas rápidas sin actualizaciones, como mínimo o máximo en rango.

```

1  struct SparseTable
2  {
3      vvi st;
4      vi lg;
5      SparseTable(&a)
6      {
7          int n=a.size(); int m=__lg(n)+1;
8          st.assign(m,vi(n));
9          lg.assign(n+1,0);
10         forn(i,2,n+1) lg[i]=lg[i/2]+1;
11         st[0]=a;
12         forn(k,1,m)
13             forn(i,0,n-(1<<k)+1)
14                 st[k][i]=min(st[k-1][i],st[k-1][i+(1<<(k-1))]);
15     }
16     int query(int l,int r)
17     {
18         int k=lg[r-l+1];
19         return min(st[k][l],st[k][r-(1<<k)+1]);
20     }
21 };

```

22.3 Fenwick Tree

```
1 struct Fenwick
2 {
3     struct ftdata {}; // Datos que guarda cada posición
4     ftdata eye;       // Elemento identidad
5     ftdata combine(ftdata a, ftdata b) {} // Combina dos datos
6
7     ll n;
8     vector<ftdata> ft;
9
10    void build(ll x)
11    {
12        n = x;
13        ft.assign(n + 1, eye);
14    }
15
16    void update(ll i, ftdata v)
17    {
18        for (++i; i <= n; i += i & -i)
19            ft[i] = combine(ft[i], v);
20    }
21
22    ftdata query(ll i)
23    {
24        ftdata res = eye;
25        for (++i; i > 0; i -= i & -i)
26            res = combine(res, ft[i]);
27        return res;
28    }
29
30    ftdata rangeQuery(ll l, ll r)
31    {
32        ftdata right = query(r);
33        if (l == 0) return right;
34        ftdata left = query(l - 1);
35        return combine(right, left);
36    }
37 };
```

22.4 Index Compression

La compresión de índices transforma valores grandes o dispersos en un rango pequeño y continuo, preservando el orden relativo. Es útil cuando los valores son grandes (por ejemplo, hasta 10^9) pero solo se usan unos pocos, permitiendo usar estructuras como Fenwick o Segment Trees sin desperdiciar memoria. Se logra ordenando los valores únicos y reemplazando cada valor original por su posición comprimida.

```
1 struct IndexCompression
2 {
3     vi vals;
4
5     void build(const vi &a)
6     {
7         vals = a;
8         sort(all(vals));
9         vals.erase(unique(all(vals)), vals.end());
10    }
11
12    ll get(ll x) { return lower_bound(all(vals), x) - vals.begin(); }
13    ll query(ll x) { return upper_bound(all(vals), x) - vals.begin(); }
14};
```

23 Arboles

```
1 struct Tree
2 {
3     int n, LOG, timer=0;
4     vvi g, up;
5     vi depth, parent, tin, tout;
6
7     Tree(const vvi &adj, int root=0)
8     {
9         g=adj; n=g.size();
10        depth.assign(n,0); parent.assign(n,-1);
11        tin.assign(n,0); tout.assign(n,0);
12        LOG=1; while((1<<LOG)<=n) LOG++;
13        up.assign(n,vi(LOG,-1));
14        dfs(root); build_lift();
15    }
16
17    void dfs(int u, int p=-1)
18    {
19        tin[u]=++timer; parent[u]=p;
20        if(p!=-1) depth[u]=depth[p]+1;
21        for(int v:g[u]) if(v!=p) dfs(v,u);
22        tout[u]=++timer;
23    }
24
25    void build_lift()
26    {
27        forn(i,n) up[i][0]=(parent[i]==-1?i:parent[i]);
28        for(int j=1;j<LOG;j++) forn(i,n)
29            up[i][j]=up[up[i][j-1]][j-1];
30    }
31};
```

23.1 Binary Tree Traversals

Recorridos clásicos de árbol: preorden, inorden y postorden.

```

1 void preorder(Tree &t,int u,int p=-1)
2 {
3     cout<<u<<" ";
4     for(int v:t.g[u]) if(v!=p) preorder(t,v,u);
5 }
6
7 void inorder(Tree &t,int u,int p=-1)
8 {
9     if(t.g[u].empty()){ cout<<u<<" "; return; }
10    bool first=true;
11    for(int v:t.g[u]) if(v!=p){
12        if(first){ inorder(t,v,u); first=false; }
13        else cout<<u<<" ", inorder(t,v,u);
14    }
15    if(first) cout<<u<<" ";
16 }
17
18 void postorder(Tree &t,int u,int p=-1)
19 {
20     for(int v:t.g[u]) if(v!=p) postorder(t,v,u);
21     cout<<u<<" ";
22 }
23
24 // Uso:
25 int main()
26 {
27     vvi g={{1,2},{0,3,4},{0},{1},{1}};
28     Tree t(g);
29     cout<<"Preorder: "; preorder(t,0); cout<<"\n";
30     cout<<"Inorder: "; inorder(t,0); cout<<"\n";
31     cout<<"Postorder: "; postorder(t,0); cout<<"\n";
32 }

```

24 Calculating Tree Diameter

Usa dos DFS para obtener el diámetro del árbol (camino más largo entre dos nodos) en $O(n)$.

```
1 int tree_diameter(Tree &t)
2 {
3     function<pair<int,int>>(int,int)> farthest=[&](int u,int p)
4     {
5         pair<int,int> best={0,u};
6         for(int v:t.g[u]) if(v!=p)
7         {
8             auto cur=farthest(v,u);
9             cur.first++;
10            best=max(best,cur);
11        }
12        return best;
13    };
14    auto a=farthest(0,-1);
15    auto b=farthest(a.second,-1);
16    return b.first;
17 }
18
19 int main()
20 {
21     vvi g={{1},{0,2,3},{1},{1,4},{3}};
22     Tree t(g);
23     cout<<"Diameter: " <<tree_diameter(t)<<"\n";
24 }
```

24.1 All longest paths

Calcula para cada nodo la distancia máxima a cualquier otro nodo (reroot DP clásico). Complejidad $O(n)$.

```
1 vi all_longest_path(Tree &t)
2 {
3     vi max1(t.n), max2(t.n), up(t.n);
4
5     function<void(int, int)> dfs_down = [&](int s, int p)
6     {
7         max1[s] = max2[s] = 0;
8         for(int u: t.g[s]) if(u != p)
9         {
10             dfs_down(u, s);
11             int len = max1[u] + 1;
12             if(len > max1[s]) swap(len, max1[s]);
13             if(len > max2[s]) max2[s] = len;
14         }
15     };
16
17     function<void(int, int)> dfs_up = [&](int s, int p)
18     {
19         for(int u: t.g[s]) if(u != p)
20         {
21             if(max1[u] + 1 == max1[s]) up[u] = max(up[s], max2[s]) + 1;
22             else up[u] = max(up[s], max1[s]) + 1;
23             dfs_up(u, s);
24         }
25     };
26
27     dfs_down(0, -1); up[0] = 0; dfs_up(0, -1);
28     vi ans(t.n);
29     forn(i, t.n) ans[i] = max(max1[i], up[i]);
30     return ans;
31 }
32
33 int main(){
34     vvi g = {{1, 2}, {0, 3, 4}, {0}, {1}, {1}};
35     Tree t(g);
36     vi d = all_longest_path(t);
37     forn(i, t.n) cout << "Longest from " << i << ": " << d[i] << "\n";
38 }
```

24.2 Lowest Common Ancestor (LCA)

Encuentra el ancestro común más bajo entre dos nodos a y b usando binary lifting.

```
1 int LCA(Tree &t, int a, int b)
2 {
3     auto is_ancestor=[&](int u, int v)
4     {
5         return t.tin[u]<=t.tin[v] && t.tout[u]>=t.tout[v];
6     };
7     if(is_ancestor(a,b)) return a;
8     if(is_ancestor(b,a)) return b;
9
10    for(int j=t.LOG-1; j>=0; j--)
11        if(!is_ancestor(t.up[a][j], b))
12            a=t.up[a][j];
13    return t.parent[a];
14 }
15
16 // Uso:
17 int main()
18 {
19     vvi g={{1,2},{0,3,4},{0},{1},{1}};
20     Tree t(g);
21     cout<<"LCA(3,4): "<<LCA(t,3,4)<<"\n";
22 }
```

24.3 Balanced Factor

Calcula el factor de balance (altura izquierda - altura derecha) para verificar si el árbol está equilibrado. (No usa rotaciones, solo cálculo conceptual.)

```
1 int balance_factor(Tree &t, int u)
2 {
3     function<int(int,int)> height=[&](int x, int p)
4     {
5         int h=0;
6         for(int v:t.g[x]) if(v!=p)
7             h=max(h, 1+height(v,x));
8         return h;
9     };
10    vi hs;
11    for(int v:t.g[u]) hs.push_back(height(v,u));
12    if(hs.empty()) return 0;
13    sort(all(hs), greater<int>());
14    int l=hs[0], r=hs.size()-1?hs[1]:0;
15    return l-r;
16 }
17
18 // Uso:
19 int main()
20 {
21     vvi g={{1,2},{0},{0}};
22     Tree t(g);
23     cout<<"Balance(0): "<<balance_factor(t,0)<<"\n";
24 }
```


25 Trie

```
1 #define ALPH 26
2
3 struct Trie
4 {
5     vvi nxt
6     vb end
7
8     Trie()
9     {
10         nxt.push_back(vi(ALPH, -1))
11         end.push_back(false)
12     }
13
14     void insert(string s)
15     {
16         int u=0
17         for(char c:s)
18         {
19             int i=c-'a'
20             if(nxt[u][i]==-1)
21             {
22                 nxt[u][i]=nxt.size()
23                 nxt.push_back(vi(ALPH, -1))
24                 end.push_back(false)
25             }
26             u=nxt[u][i]
27         }
28         end[u]=true
29     }
30
31     bool find(string s)
32     {
33         int u=0
34         for(char c:s)
35         {
36             int i=c-'a'
37             if(nxt[u][i]==-1) return false
38             u=nxt[u][i]
39         }
40         return end[u]
41     }
42
43     void erase(string s)
44     {
45         function<bool(int,int)> del=[&](int u, int d)
46         {
47             if(u==-1) return true
48             if(d==s.size()) end[u]=false
49             else
50             {
51                 int i=s[d]-'a'
52                 if(del(nxt[u][i], d+1)) nxt[u][i]=-1
53             }
54             bool empty=true
55             forn(i, ALPH) if(nxt[u][i]!=-1) empty=false
56             return !end[u] && empty
57         }
58         del(0, 0)
59     }
60
61     void print()
62     {
```

```

63     string cur
64     function<void(int)> dfs=[&](int u)
65     {
66         if(end[u]) cout<<cur<<endl
67         forn(i,ALPH)
68         {
69             if(nxt[u][i]!=-1)
70             {
71                 cur.push_back('a'+i)
72                 dfs(nxt[u][i])
73                 cur.pop_back()
74             }
75         }
76     }
77     dfs(0)
78 }
79
80 void clear()
81 {
82     nxt.assign(1,vi(ALPH,-1))
83     end.assign(1,false)
84 }
85 }

```

26 Rootear un arbol desde el centro

```
1  vi treeCenters(vvi& g)
2  {
3      int n = g.size();
4      vi degree(n, 0), leaves;
5
6      forn(i, 0, n)
7      {
8          degree[i] = g[i].size();
9          if (degree[i] == 0 || degree[i] == 1)
10             {
11                 leaves.push_back(i);
12                 degree[i] = 0;
13             }
14     }
15
16     int count = leaves.size();
17
18     while (count < n) {
19         vector<int> new_leaves;
20         for (int node : leaves)
21             {
22                 for (int neighbor : g[node])
23                     {
24                         degree[neighbor] = degree[neighbor] - 1;
25                         if (degree[neighbor] == 1)
26                             new_leaves.push_back(neighbor);
27                         degree[node] = 0;
28                     }
29             }
30         count += new_leaves.size();
31         leaves = new_leaves;
32     }
33
34     return leaves; // Centros (center) del árbol
35 }
36
37 void dfs(vvi &g, vb& visited, vvi& rootedTree, int node)
38 {
39     if (!visited[node])
40     {
41         visited[node] = true;
42         for (int neighbor : g[node])
43             {
44                 if (!visited[neighbor])
45                     {
46                         rootedTree[node].push_back(neighbor);
47                         dfs(g, visited, rootedTree, neighbor);
48                     }
49             }
50     }
51 }
52
53 int main()
54 {
55     int n = 6;
56     vvi graph(n);
57     // Agregar las conexiones del grafo
58     vi centers = treeCenters(graph);
59     cout << "Centro(s) del árbol: ";
60     for (int center : centers) {
61         cout << center << " ";
62     }
```

```

63     cout << endl;
64
65     // Elegir el primer centro como raíz
66     int root = centers[0];
67
68     vvi rootedTree(n);
69     vb visited(n, false);
70
71     dfs(graph, visited, rootedTree, root);
72
73     cout << "Grafo enraizado desde el centro:" << endl;
74     forn(i, 0, n)
75     {
76         cout << i << ": ";
77         for (int neighbor : rootedTree[i])
78         {
79             cout << neighbor << " ";
80         }
81         cout << endl;
82     }
83
84     return 0;
85 }

```

27 Identificar dos arboles isomorficos

```

1
2 using namespace std;
3
4 // Función para realizar un recorrido DFS desde el nodo raíz central
5 void dfs(int u, int parent, vvi &graph, vi& code)
6 {
7     code.push_back(1); // Etiqueta de apertura
8     for (int v : graph[u])
9     {
10         if (v != parent)
11             dfs(v, u, graph, code);
12     }
13     code.push_back(0); // Etiqueta de cierre
14 }
15
16 // Función para generar el código AHU de un grafo en forma de lista de adyacencia
17 vi AHU_Code(vvi& graph, int root) {
18     vector<int> code;
19     dfs(root, -1, graph, code);
20     return code;
21 }
22
23
24 int main() {
25     // Definir los grafos como listas de adyacencia
26     vvi graph1 = {{1, 2}, {0, 3}, {0, 3}, {1, 2}};
27     vvi graph2 = {{1, 2, 3}, {0, 4}, {0, 4}, {0, 4}, {1, 2, 3}};
28
29     // Supongamos que el nodo raíz central es el nodo 0 en ambos grafos
30     int root = 0;
31
32     // Generar códigos AHU para ambos grafos
33     vvi code1 = AHU_Code(graph1, root);
34     vvi code2 = AHU_Code(graph2, root);
35
36     // Verificar si los códigos son iguales

```

```

37     if (code1 == code2) {
38         cout << "Los grafos son isomorfos." << endl;
39     } else {
40         cout << "Los grafos no son isomorfos." << endl;
41     }
42
43     return 0;
44 }

```

28 Algoritmo de Tarjan para Componentes Fuertemente Conexos

```

1  void tarjan(int u, vvi& grafo, vi& ids, vi& bajos, stack<int>& pila, vb& enPila, int& idActual,
2  vvi& componentes) {
3      ids[u] = bajos[u] = idActual++;
4      pila.push(u);
5      enPila[u] = true;
6
7      for (int v : grafo[u])
8      {
9          if (ids[v] == -1)
10             {
11                 tarjan(v, grafo, ids, bajos, pila, enPila, idActual, componentes);
12             }
13             if (enPila[v])
14                 bajos[u] = min(bajos[u], bajos[v]);
15
16     }
17
18     if (ids[u] == bajos[u])
19     {
20         vi componente;
21         while (true)
22         {
23             int nodo = pila.top();
24             pila.pop();
25             enPila[nodo] = false;
26             componente.push_back(nodo);
27             if (nodo == u) break;
28         }
29         componentes.push_back(componente);
30     }
31 }
32
33 vvi componentesConexos(vvi& grafo) {
34     int n = grafo.size();
35     vi ids(n, -1), bajos(n, -1);
36     vb enPila(n, false);
37     stack<int> pila;
38     int idActual = 0;
39     vvi componentes;
40
41     for (int i = 0; i < n; i++)
42     {
43         if (ids[i] == -1)
44         {
45             tarjan(i, grafo, ids, bajos, pila, enPila, idActual, componentes);
46         }
47     }
48
49     return componentes;
50 }

```

28.1 Sucesiones

Números de Fibonacci: $F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$

Números de Lucas: $L(n) = L(n-1) + L(n-2)$, $L(0) = 2$, $L(1) = 1$

Números de Pell: $P(n) = 2P(n-1) + P(n-2)$, $P(0) = 0$, $P(1) = 1$

Números de Mersenne: $M_n = 2^n - 1$, n primo

Números Poligonales: $P(s, n) = \frac{(s-2)n^2 - (s-4)n}{2}$, $s \geq 3$

28.2 Sumatorias

Suma Gaussiana: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Números Triangulares: $T(n) = \frac{n(n+1)}{2}$, $T(1) = 1$

Números Cuadrados: $S(n) = n^2$, $S(1) = 1$

Números Triangulares Centrales: $TC(n) = \frac{n(n+1)(2n+1)}{6}$, $TC(1) = 1$

Números Triangulares Tetraédricos: $Te(n) = \frac{n(n+1)(n+2)}{6}$

Potencias: $a^n, 2^n, 3^n, \dots$

Factoriales: $n! = 1 \cdot 2 \cdot \dots \cdot n$, $0! = 1$

28.3 Combinatoria

Números de Catalan: $C(n) = \frac{(2n)!}{(n+1)!n!}$, $C(0) = 1$

Números de Stirling de Segundo Tipo: $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$

Números de Bell: $B(n) = \sum_{k=0}^n S(n, k)$

28.4 Números Especiales

Números Primos: no hay fórmula cerrada

Números Armónicos: $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$

Números Perfectos: $\sigma(n) = 2n$ (para números perfectos)

Números de Fermat: $F_n = 2^{2^n} + 1$

s de Fermat:} & F_n = 2^{2^n} + 1 \end{array} \end{array}

28.4.1 Divisibilidad

Número	Condición	Número	Condición
1	Divisible entre 1	51	Divisible entre 3 y 17
2	Último dígito par	52	Últimos 2 dígitos divisible entre 4
3	Suma de dígitos divisible entre 3	53	Divisible entre 53
4	Últimos 2 dígitos divisible entre 4	54	Divisible entre 2, 3 y 9
5	Último dígito 0 o 5	55	Divisible entre 5 y 11
6	Divisible entre 2 y 3	56	Últimos 3 dígitos divisible entre 8
7	Regla del 7	57	Divisible entre 3 y 19
8	Últimos 3 dígitos divisible entre 8	58	Divisible entre 2 y 29
9	Suma de dígitos divisible entre 9	59	Divisible entre 59
10	Último dígito 0	60	Divisible entre 2, 3 y 10
11	Diferencia pares/impares divisible entre 11	61	Divisible entre 61
12	Divisible entre 3 y 4	62	Divisible entre 2 y 31
13	Regla del 13	63	Divisible entre 3, 7 y 9
14	Divisible entre 2 y 7	64	Últimos 6 dígitos divisible entre 64
15	Divisible entre 3 y 5	65	Divisible entre 5 y 13
16	Últimos 4 dígitos divisible entre 16	66	Divisible entre 2, 3 y 11
17	Divisible entre 17	67	Divisible entre 67
18	Divisible entre 2 y 9	68	Últimos 2 dígitos divisible entre 4
19	Divisible entre 19	69	Divisible entre 3 y 23
20	Divisible entre 4 y 5	70	Divisible entre 2, 5 y 7
21	Divisible entre 3 y 7	71	Divisible entre 71
22	Divisible entre 2 y 11	72	Divisible entre 3, 4 y 9
23	Divisible entre 23	73	Divisible entre 73
24	Divisible entre 3 y 8	74	Divisible entre 2 y 37
25	Termina en 00 o múltiplo de 25/50/75	75	Divisible entre 3 y 25
26	Divisible entre 2 y 13	76	Últimos 2 dígitos divisible entre 4
27	Divisible entre 3 y 9	77	Divisible entre 7
28	Divisible entre 4 y 7	78	Divisible entre 2, 3 y 13
29	Divisible entre 29	79	Divisible entre 79
30	Divisible entre 2, 3 y 5	80	Últimos 2 dígitos divisible entre 16
31	Divisible entre 31	81	Divisible entre 3 y 27
32	Últimos 5 dígitos divisible entre 32	82	Divisible entre 2 y 41
33	Divisible entre 3 y 11	83	Divisible entre 83
34	Divisible entre 2 y 17	84	Divisible entre 2, 3, 4 y 7
35	Divisible entre 5 y 7	85	Divisible entre 5 y 17
36	Divisible entre 4 y 9	86	Divisible entre 2 y 43
37	Divisible entre 37	87	Divisible entre 3 y 29
38	Divisible entre 2 y 19	88	Últimos 2 dígitos divisible entre 8
39	Divisible entre 3 y 13	89	Divisible entre 89
40	Divisible entre 4 y 10	90	Divisible entre 2, 3 y 9
41	Divisible entre 41	91	Divisible entre 7 y 13
42	Divisible entre 2, 3 y 7	92	Últimos 2 dígitos divisible entre 4
43	Divisible entre 43	93	Divisible entre 3 y 31
44	Últimos 2 dígitos divisible entre 4	94	Divisible entre 2 y 47
45	Divisible entre 3 y 5	95	Divisible entre 5 y 19
46	Divisible entre 2 y 23	96	Divisible entre 3, 4 y 8
47	Divisible entre 47	97	Divisible entre 97
48	Divisible entre 3 y 16	98	Divisible entre 2, 7 y 14
49	Divisible entre 7	99	Divisible entre 3 y 11
50	Divisible entre 2	100	Últimos 2 dígitos 0

29 Meet in the Middle

El algoritmo “Meet in the Middle” es una técnica de optimización que reduce la complejidad computacional de problemas de búsqueda exhaustiva mediante la partición del espacio de soluciones en dos mitades independientes. Su fundamento radica en el principio divide y vencerás: en lugar de explorar todas las 2^n combinaciones posibles en un problema, se divide el conjunto de n elementos en dos subconjuntos de tamaño $n/2$, generando todas las configuraciones posibles para cada mitad ($2^{(n/2)}$ cada una), almacenando los resultados de la primera mitad en una estructura de datos eficiente (típicamente una tabla hash o árbol de búsqueda), y luego explorando la segunda mitad buscando complementos que satisfagan las condiciones del problema.

```
1 // Función que implementa la parte "izquierda" del algoritmo
2 void generateSubsetsLeft(vector<int>& nums, int index, int sum, map<int, int>& leftSumCounts) {
3     if (index == nums.size() / 2) {
4         leftSumCounts[sum]++;
5         return;
6     }
7
8     // Incluir el elemento actual en el subconjunto
9     generateSubsetsLeft(nums, index + 1, sum + nums[index], leftSumCounts);
10
11    // No incluir el elemento actual en el subconjunto
12    generateSubsetsLeft(nums, index + 1, sum, leftSumCounts);
13 }
14
15 // Función que implementa la parte "derecha" del algoritmo
16 int countSubsetsRight(vector<int>& nums, int index, int sum, map<int, int>& leftSumCounts) {
17     if (index == nums.size()) {
18         int complement = -sum; // Calcula el complemento necesario para alcanzar el objetivo
19         if (leftSumCounts.find(complement) != leftSumCounts.end()) {
20             return leftSumCounts[complement];
21         }
22         return 0;
23     }
24
25     // Incluir el elemento actual en el subconjunto
26     int includeCount = countSubsetsRight(nums, index + 1, sum + nums[index], leftSumCounts);
27
28     // No incluir el elemento actual en el subconjunto
29     int excludeCount = countSubsetsRight(nums, index + 1, sum, leftSumCounts);
30
31     return includeCount + excludeCount;
32 }
33
34 int main() {
35     vector<int> nums = {1, 2, 3, 4, 5};
36     int targetSum = 6;
37
38     map<int, int> leftSumCounts;
39     generateSubsetsLeft(nums, 0, 0, leftSumCounts);
40
41     int totalSubsets = countSubsetsRight(nums, nums.size() / 2, 0, leftSumCounts);
42
43     cout << "Número de subconjuntos con suma igual a " << targetSum << ": " << totalSubsets <<
44         endl;
45     return 0;
46 }
```

30 Algoritmo de Mo

El algoritmo de Mo es una técnica de optimización para responder múltiples consultas sobre subarreglos de manera eficiente mediante reordenamiento inteligente. Divide el arreglo en bloques de tamaño $\sqrt{n}$ y ordena las consultas

primero por el bloque inicial y luego por el índice final, permitiendo que el puntero se mueva de forma incremental.

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int block_size;
7
8 // Estructura de datos que almacena los valores y permite realizar las operaciones
9 struct DataStructure {
10     vector<int> data;
11
12     void add(int idx) {
13         // TODO: Implementar la lógica para agregar un valor en el índice idx
14     }
15
16     void remove(int idx) {
17         // TODO: Implementar la lógica para eliminar un valor en el índice idx
18     }
19
20     int get_answer() {
21         // TODO: Implementar la lógica para calcular la respuesta actual de la estructura de datos
22         return 0; // Cambiar 0 por el valor correcto
23     }
24 };
25
26 vector<int> mo_s_algorithm(vector<Query> queries) {
27     vector<int> answers(queries.size());
28     sort(queries.begin(), queries.end());
29
30     DataStructure ds;
31
32     int cur_l = 0;
33     int cur_r = -1;
34
35     for (Query q : queries) {
36         while (cur_l > q.l) {
37             cur_l--;
38             ds.add(cur_l);
39         }
40         while (cur_r < q.r) {
41             cur_r++;
42             ds.add(cur_r);
43         }
44         while (cur_l < q.l) {
45             ds.remove(cur_l);
46             cur_l++;
47         }
48         while (cur_r > q.r) {
49             ds.remove(cur_r);
50             cur_r--;
51         }
52         answers[q.idx] = ds.get_answer();
53     }
54     return answers;
55 }
56
57 int main() {
58     // Ejemplo de uso del algoritmo de Mo's
59     block_size = 3; // Tamaño del bloque, ajusta según tus necesidades
60
61     vector<Query> queries = {
62         {0, 4, 0},
63         {2, 6, 1},
```

```

64     {1, 5, 2},
65 };
66
67 vector<int> results = mo_s_algorithm(queries);
68
69 for (int answer : results) {
70     cout << answer << " ";
71 }
72
73 return 0;
74 }

```

31 Algoritmo de Kadane

El algoritmo de Kadane resuelve el problema del subarreglo de suma máxima en tiempo lineal $O(n)$ mediante programación dinámica. Mantiene dos variables: la suma máxima terminando en la posición actual y la suma máxima global encontrada. Itera el arreglo una vez, decidiendo en cada paso si extender el subarreglo actual o iniciar uno nuevo, demostrando que la solución óptima puede construirse incrementalmente sin explorar todas las $O(n^2)$ posibilidades.

```

1  #include <iostream>
2  #include <algorithm> // Para std::max
3  #include <climits>   // Para INT_MIN
4
5  // Función para encontrar la suma máxima del subarreglo
6  int kadane(int arr[], int n) {
7      int max_so_far = INT_MIN; // Inicializa con el menor entero posible
8      int max_ending_here = 0;  // Suma máxima hasta el elemento actual
9
10     for (int i = 0; i < n; i++) {
11         // Suma el elemento actual a la suma actual
12         max_ending_here = max_ending_here + arr[i];
13
14         // Si la suma actual es mayor que la suma global, actualiza la suma global
15         if (max_so_far < max_ending_here) {
16             max_so_far = max_ending_here;
17         }
18
19         // Si la suma actual es negativa, reiníciala a 0
20         if (max_ending_here < 0) {
21             max_ending_here = 0;
22         }
23     }
24     // Este código puede retornar 0 para arrays con solo números negativos,
25     // Para manejar correctamente arrays con solo números negativos, se necesita una comprobación
26     // adicional.
27     // En este caso, el valor máximo es el número menos negativo.
28     // Si `max_so_far` sigue siendo `INT_MIN` después del bucle (lo que puede pasar si el array
29     // está vacío o solo tiene números negativos),
30     // y `max_ending_here` nunca fue mayor que 0, es necesario manejar el caso de los números
31     // negativos.
32     // Por lo tanto, se debe encontrar el elemento máximo del array si `max_so_far` es negativo.
33     // Por simplicidad, asumiremos aquí que el arreglo contiene al menos un elemento no negativo.
34
35     // Esta versión es para la suma máxima de un subarreglo.
36     // Si el arreglo puede contener solo números negativos, un enfoque más completo:
37
38     int max_sum_negative_only = arr[0];
39     for(int i = 1; i < n; i++) {
40         if(arr[i] > max_sum_negative_only) {
41             max_sum_negative_only = arr[i];
42         }
43     }
44 }

```

```

41     if (max_so_far < 0) return max_sum_negative_only;
42
43     return max_so_far;
44 }
45
46 // Función principal para probar el algoritmo
47 int main() {
48     int arr[] = {-2, -3, 4, -1, -2, 1, 5, -3};
49     int n = sizeof(arr) / sizeof(arr[0]);
50     std::cout << "El subarreglo con la suma maxima es: " << kadane(arr, n) << std::endl; //
        Salida: 7
51     return 0;
52 }

```

32 Binary Lifting

Binary Lifting es una técnica de preprocesamiento para responder consultas de ancestros en árboles en tiempo logarítmico. Precalcula para cada nodo sus ancestros a distancias potencias de dos ($2^0, 2^1, 2^2, \dots$) mediante programación dinámica, permitiendo alcanzar cualquier ancestro k -ésimo mediante descomposición binaria de k . Con complejidad $O(n \log n)$ de preprocesamiento y $O(\log n)$ por consulta, es fundamental para implementar Lowest Common Ancestor (LCA) eficientemente.

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4
5  const int MAXN = 100005; // Número máximo de nodos
6  const int LOGN = 18;    // Logaritmo base 2 del número máximo de nodos
7
8  std::vector<int> adj[MAXN];
9  int parent[MAXN];
10 int dp[MAXN][LOGN];
11 int depth[MAXN];
12
13 void dfs(int u, int p, int d) {
14     parent[u] = p;
15     depth[u] = d;
16     dp[u][0] = p; // El  $2^0$ -ésimo ancestro es el padre
17
18     // Precalculamos los ancestros para todos los nodos
19     for (int i = 1; i < LOGN; ++i) {
20         dp[u][i] = dp[dp[u][i - 1]][i - 1];
21     }
22
23     for (int v : adj[u]) {
24         if (v != p) {
25             dfs(v, u, d + 1);
26         }
27     }
28 }
29
30 // Función para encontrar el k-ésimo ancestro de un nodo u
31 int get_kth_ancestor(int u, int k) {
32     // Si k es mayor que la profundidad del nodo, no existe el ancestro
33     if (k > depth[u]) {
34         return -1; // Indicador de que no hay ancestro
35     }
36
37     // Recorremos los bits de k
38     for (int i = 0; i < LOGN; ++i) {
39         if ((k >> i) & 1) { // Si el i-ésimo bit es 1
40             u = dp[u][i];

```

```

41     }
42 }
43 return u;
44 }
45
46 int main() {
47     // Ejemplo de uso: Construir un árbol y precalcular
48     int n; // número de nodos
49     std::cin >> n;
50
51     // Leer aristas (ejemplo: arista entre u y v)
52     // ...
53
54     // Llama a la DFS para precalcular dp y depth
55     dfs(1, 0, 0); // Asumiendo que la raíz es 1 y su padre es 0
56
57     int u, k;
58     std::cin >> u >> k;
59
60     int ancestor = get_kth_ancestor(u, k);
61
62     if (ancestor != -1) {
63         std::cout << "El " << k << "-ésimo ancestro de " << u << " es " << ancestor << std::endl;
64     } else {
65         std::cout << "No tiene un " << k << "-ésimo ancestro." << std::endl;
66     }
67
68     return 0;
69 }

```

33 Knuth-Morris-Pratt KMP

El algoritmo KMP busca patrones en texto con complejidad $O(n + m)$ mediante un preprocesamiento que evita comparaciones redundantes. Construye un arreglo de prefijos que almacena para cada posición la longitud del mayor prefijo propio que es también sufijo. Durante la búsqueda, cuando ocurre un desajuste, el algoritmo utiliza esta información para desplazar el patrón inteligentemente sin retroceder en el texto, eliminando el comportamiento $O(nm)$ del enfoque ingenuo.

```

1  vi computeLPSArray(const string& pattern)
2  {
3      int m = pattern.length();
4      vector<int> lps(m, 0);
5      int len = 0;
6
7      for (int i = 1; i < m; i++)
8      {
9          while (len > 0 && pattern[i] != pattern[len]) len = lps[len - 1];
10         if (pattern[i] == pattern[len]) len++;
11         lps[i] = len;
12     }
13
14     return lps;
15 }
16
17 vi KMPSearch(const string& text, const string& pattern) {
18     vi indices, lps = computeLPSArray(pattern);
19     int n = text.length(), m = pattern.length(), i = 0, j = 0;
20
21     while (i < n)
22     {
23         if (pattern[j] == text[i]) i++, j++;
24         if (j == m) indices.push_back(i - j), j = lps[j - 1];

```

```

25     else if (i < n && pattern[j] != text[i]) j = j ? lps[j - 1] : i++;
26 }
27
28 return indices;
29 }

```

34 Algoritmo de Manacher

El algoritmo de Manacher encuentra todos los palíndromos en una cadena en tiempo lineal $O(n)$. Utiliza simetría para evitar recalcular información ya conocida, manteniendo el centro y radio del palíndromo más a la derecha encontrado. Transforma la cadena insertando caracteres especiales para manejar uniformemente palíndromos de longitud par e impar, siendo significativamente más eficiente que el enfoque $O(n)$ de expansión desde el centro.

```

1  string manacher(const string& s) {
2      string T = "^#";
3      for (char c : s) {
4          T += c;
5          T += '#';
6      }
7      T += '$';
8
9      int n = T.size();
10     vector<int> P(n, 0);
11     int C = 0, R = 0;
12
13     for (int i = 1; i < n - 1; i++) {
14         int i_mirror = 2 * C - i;
15         if (R > i) {
16             P[i] = min(R - i, P[i_mirror]);
17         }
18
19         while (T[i + 1 + P[i]] == T[i - 1 - P[i]]) {
20             P[i]++;
21         }
22
23         if (i + P[i] > R) {
24             C = i;
25             R = i + P[i];
26         }
27     }
28
29     int maxLen = 0;
30     int centerIndex = 0;
31     for (int i = 1; i < n - 1; i++) {
32         if (P[i] > maxLen) {
33             maxLen = P[i];
34             centerIndex = i;
35         }
36     }
37
38     int start = (centerIndex - maxLen) / 2;
39     return s.substr(start, maxLen);
40 }

```

35 Longest Common Subsequence (LCS):

LCS encuentra la subsecuencia común más larga entre dos secuencias mediante programación dinámica con complejidad $O(nm)$. La relación de recurrencia establece que $LCS[i][j] = LCS[i-1][j-1] + 1$ si los caracteres coinciden, o $\max(LCS[i-1][j], LCS[i][j-1])$ en caso contrario. Esta técnica tiene aplicaciones en bioinformática para alineamiento de secuencias, control de versiones y detección de similitud textual.

```

1  string findLCS(const string& X, const string& Y) {
2      int m = X.length();
3      int n = Y.length();
4
5      vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
6
7      for (int i = 1; i <= m; i++) {
8          for (int j = 1; j <= n; j++) {
9              if (X[i - 1] == Y[j - 1]) {
10                 dp[i][j] = dp[i - 1][j - 1] + 1;
11             } else {
12                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
13             }
14         }
15     }
16
17     int lcs_length = dp[m][n];
18     string lcs(lcs_length, ' ');
19
20     int i = m, j = n;
21     while (i > 0 && j > 0) {
22         if (X[i - 1] == Y[j - 1]) {
23             lcs[--lcs_length] = X[i - 1];
24             i--;
25             j--;
26         } else if (dp[i - 1][j] > dp[i][j - 1]) {
27             i--;
28         } else {
29             j--;
30         }
31     }
32
33     return lcs;
34 }

```

36 Rootear un árbol

Rootear un árbol consiste en designar un nodo como raíz y establecer relaciones padre-hijo mediante DFS o BFS desde ese nodo. Este proceso transforma un árbol no dirigido en un árbol dirigido con jerarquía, facilitando operaciones como cálculo de ancestros, profundidades y aplicación de algoritmos de programación dinámica en árboles. La complejidad es $O(n)$ y la elección de la raíz puede optimizarse usando el centroide para minimizar la altura.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  vector<int> treeCenters(vector<vector<int>>& g) {
7      int n = g.size();
8      vector<int> degree(n, 0);
9      vector<int> leaves;
10
11     for (int i = 0; i < n; i++) {
12         degree[i] = g[i].size();
13         if (degree[i] == 0 || degree[i] == 1) {
14             leaves.push_back(i);
15             degree[i] = 0;
16         }
17     }
18
19     int count = leaves.size();
20

```



```

21 while (count < n) {
22     vector<int> new_leaves;
23     for (int node : leaves) {
24         for (int neighbor : g[node]) {
25             degree[neighbor] = degree[neighbor] - 1;
26             if (degree[neighbor] == 1) {
27                 new_leaves.push_back(neighbor);
28             }
29             degree[node] = 0;
30         }
31     }
32     count += new_leaves.size();
33     leaves = new_leaves;
34 }
35
36 return leaves; // Centros (center) del árbol
37 }
38
39 void dfs(vector<vector<int>>& g, vector<bool>& visited, vector<vector<int>>& rootedTree, int node)
40 {
41     if (!visited[node]) {
42         visited[node] = true;
43         for (int neighbor : g[node]) {
44             if (!visited[neighbor]) {
45                 rootedTree[node].push_back(neighbor);
46                 dfs(g, visited, rootedTree, neighbor);
47             }
48         }
49     }
50 }
51
52 int main() {
53     // Definir el grafo no dirigido como una lista de adyacencia
54     int n = 6;
55     vector<vector<int>> graph(n);
56
57     // Agregar las conexiones del grafo
58     graph[0].push_back(1);
59     graph[1].push_back(0);
60     graph[1].push_back(2);
61     graph[2].push_back(1);
62     graph[2].push_back(3);
63     graph[3].push_back(2);
64     graph[3].push_back(4);
65     graph[4].push_back(3);
66     graph[4].push_back(5);
67     graph[5].push_back(4);
68
69     vector<int> centers = treeCenters(graph);
70
71     cout << "Centro(s) del árbol: ";
72     for (int center : centers) {
73         cout << center << " ";
74     }
75     cout << endl;
76
77     // Elegir el primer centro como raíz
78     int root = centers[0];
79
80     vector<vector<int>> rootedTree(n);
81     vector<bool> visited(n, false);
82
83     dfs(graph, visited, rootedTree, root);
84
85     cout << "Grafo enraizado desde el centro:" << endl;

```

```

85     for (int i = 0; i < n; i++) {
86         cout << i << ": ";
87         for (int neighbor : rootedTree[i]) {
88             cout << neighbor << " ";
89         }
90         cout << endl;
91     }
92
93     return 0;
94 }

```

37 Identificar dos arboles isomorficos

Determinar si dos árboles son isomorfos implica verificar si existe una biyección que preserve adyacencias. El algoritmo *AHU* (*Aho, Hopcroft, Ullman*) resuelve esto en $O(n)$ mediante hashing canónico: asigna etiquetas únicas a subárboles desde las hojas hacia la raíz, creando una representación invariante bajo isomorfismo. Dos árboles son isomorfos si sus representaciones canónicas coinciden, siendo útil en detección de patrones estructurales.

```

1  using namespace std;
2
3  // Función para realizar un recorrido DFS desde el nodo raíz central
4  void DFS(int u, int parent, vector<vector<int>>& graph, vector<int>& code) {
5      code.push_back(1); // Etiqueta de apertura
6      for (int v : graph[u]) {
7          if (v != parent) {
8              DFS(v, u, graph, code);
9          }
10     }
11     code.push_back(0); // Etiqueta de cierre
12 }
13
14 // Función para generar el código AHU de un grafo en forma de lista de adyacencia
15 vector<int> AHU_Code(vector<vector<int>>& graph, int root) {
16     vector<int> code;
17     DFS(root, -1, graph, code);
18     return code;
19 }
20
21 // Función para verificar si dos códigos AHU son iguales
22 bool areCodesEqual(const vector<int>& code1, const vector<int>& code2) {
23     return code1 == code2;
24 }
25
26 int main() {
27     // Definir los grafos como listas de adyacencia
28     vector<vector<int>> graph1 = {{1, 2}, {0, 3}, {0, 3}, {1, 2}};
29     vector<vector<int>> graph2 = {{1, 2, 3}, {0, 4}, {0, 4}, {0, 4}, {1, 2, 3}};
30
31     // Supongamos que el nodo raíz central es el nodo 0 en ambos grafos
32     int root = 0;
33
34     // Generar códigos AHU para ambos grafos
35     vector<int> code1 = AHU_Code(graph1, root);
36     vector<int> code2 = AHU_Code(graph2, root);
37
38     // Verificar si los códigos son iguales
39     if (areCodesEqual(code1, code2)) {
40         cout << "Los grafos son isomorfos." << endl;
41     } else {
42         cout << "Los grafos no son isomorfos." << endl;
43     }
44 }

```

```

45     return 0;
46 }

```

38 Algoritmo de Tarjan para Componentes Fuertemente Conexas

El algoritmo de Tarjan identifica componentes fuertemente conexas en grafos dirigidos en $O(V + E)$ mediante DFS con un solo recorrido. Utiliza una pila y dos arreglos: tiempos de descubrimiento y valores “low” que rastrean el ancestro más bajo alcanzable. Cuando un nodo tiene $low[v] = disc[v]$, identifica la raíz de una componente fuertemente conexas. Este algoritmo es fundamental para análisis de conectividad, detección de ciclos y problemas de satisfacibilidad.

```

1  #include <iostream>
2  #include <vector>
3  #include <stack>
4  #include <algorithm>
5
6  using namespace std;
7
8  void tarjan(int u, vector<vector<int>>& grafo, vector<int>& ids, vector<int>& bajos, stack<int>&
9      pila, vector<bool>& enPila, int& idActual, vector<vector<int>>& componentes) {
10     ids[u] = bajos[u] = idActual++;
11     pila.push(u);
12     enPila[u] = true;
13
14     for (int v : grafo[u]) {
15         if (ids[v] == -1) {
16             tarjan(v, grafo, ids, bajos, pila, enPila, idActual, componentes);
17         }
18         if (enPila[v]) {
19             bajos[u] = min(bajos[u], bajos[v]);
20         }
21     }
22
23     if (ids[u] == bajos[u]) {
24         vector<int> componente;
25         while (true) {
26             int nodo = pila.top();
27             pila.pop();
28             enPila[nodo] = false;
29             componente.push_back(nodo);
30             if (nodo == u) {
31                 break;
32             }
33         }
34         componentes.push_back(componente);
35     }
36 }
37
38 vector<vector<int>> encontrarComponentesFuertementeConexas(vector<vector<int>>& grafo) {
39     int n = grafo.size();
40     vector<int> ids(n, -1);
41     vector<int> bajos(n, -1);
42     vector<bool> enPila(n, false);
43     stack<int> pila;
44     int idActual = 0;
45     vector<vector<int>> componentes;
46
47     for (int i = 0; i < n; i++) {
48         if (ids[i] == -1) {
49             tarjan(i, grafo, ids, bajos, pila, enPila, idActual, componentes);
50         }
51     }

```

```

52     return componentes;
53 }
54
55 int main() {
56     int V = 5;
57     vector<vector<int>> grafo(V);
58
59     grafo[0].push_back(1);
60     grafo[1].push_back(2);
61     grafo[2].push_back(0);
62
63     grafo[3].push_back(4);
64
65     vector<vector<int>> componentes = encontrarComponentesFuertementeConexos(grafo);
66
67     cout << "Componentes fuertemente conexos:" << endl;
68     for (const vector<int>& componente : componentes) {
69         for (int nodo : componente) {
70             cout << nodo << " ";
71         }
72         cout << endl;
73     }
74
75     return 0;
76 }

```